

**IMPLEMENTASI *GRAPH RETRIEVAL*
AUGMENTED GENERATION UNTUK
MENINGKATKAN PRESISI RETRIEVAL DAN
VALIDITAS SINTAKSIS PADA KONFIGURASI
KUBERNETES**

Laporan Tugas Akhir

Oleh

**Jihan Aurelia
18222001**



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
Maret 2026**

LEMBAR PENGESAHAN

IMPLEMENTASI *GRAPH RETRIEVAL AUGMENTED GENERATION* UNTUK MENINGKATKAN PRESISI RETRIEVAL DAN VALIDITAS SINTAKSIS PADA KONFIGURASI KUBERNETES

Laporan Tugas Akhir

Oleh

Jihan Aurelia
18222001

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Laporan Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 23 Maret 2026

Pembimbing

Dr. Ir. Dimitri Mahayana, M.Eng.

NIP. 196808091991021001

PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tugas Akhir ini adalah hasil karya saya sendiri yang dibuat dengan sebenar-benarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan laporan tugas akhir ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan laporan tugas akhir.
3. Isi laporan ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa laporan ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 23 Maret 2026

Jihan Aurelia

NIM: 18222001

PERNYATAAN PENGGUNAAN AI

Saya yang bertanda tangan di bawah ini menyatakan bahwa dalam penulisan dokumen Tugas Akhir ini, saya menggunakan alat bantu kecerdasan artifisial (AI) untuk beberapa aspek tertentu di dalam proses penulisan, seperti yang tercantum dalam tabel berikut.

No.	Jenis	Alat Bantu	Tingkat	Tempat
1	Memeriksa ejaan dan tata bahasa	Grammarly	Tinggi	Semua bab
2	Pembuatan teks	Microsoft Copilot	Sedang	Bab II hal. 15
3	Pembuatan grafik, gambar, atau ilustrasi	Tidak ada	Tidak ada	Tidak ada
4	Pencarian informasi atau referensi	...	...	...
5	Penyusunan bahan diskusi	...	...	...
6	Penyusunan kode program	...	...	...
7	Penyusunan formula atau perhitungan matematis	...	...	...
8	Penyusunan konsep desain atau algoritma	...	...	...
9	Penyusunan analisis data	...	...	...
10	Penyusunan kesimpulan atau rekomendasi	...	...	...

Bandung, 23 Maret 2026

Jihan Aurelia
NIM: 18222001

ABSTRAK

Implementasi *Graph Retrieval Augmented Generation* untuk Meningkatkan Presisi Retrieval dan Validitas Sintaksis pada Konfigurasi Kubernetes

oleh

Jihan Aurelia

18222001

Proses manual dalam rantai pasok jeruk di tingkat lapak, yang mencakup sortir, timbang, dan catat, terbukti tidak efisien, memakan waktu, dan rentan terhadap kesalahan. Meskipun solusi berbasis Internet of Things (IoT) dapat mengatasi masalah ini, arsitektur terpusat konvensional berbasis cloud justru menimbulkan masalah baru berupa latensi tinggi yang menghambat alur kerja real-time. Penelitian ini bertujuan untuk merancang dan mengimplementasikan sebuah purwarupa sistem smart scale berbasis IoT dengan pendekatan desentralisasi (edge computing) untuk meminimalkan latensi dan meningkatkan efisiensi operasional. Metode yang digunakan adalah Model Purwarupa, yang diwujudkan dalam bentuk perangkat keras dengan arsitektur prosesor ganda (Raspberry Pi 5 dan ESP32) yang mampu mengintegrasikan fungsi timbang dan analisis kualitas citra secara simultan. Hasil evaluasi menunjukkan bahwa purwarupa dengan arsitektur edge computing yang diimplementasikan mampu memberikan respons 43,5% lebih cepat (latensi rata-rata 18,05 detik) dibandingkan dengan simulasi arsitektur terpusat. Selain itu, sistem ini berhasil meningkatkan efisiensi waktu kerja secara keseluruhan sebesar 58,32% jika dibandingkan dengan metode manual konvensional. Pengujian Penerimaan Pengguna (UAT) yang melibatkan 30 partisipan juga menunjukkan penerimaan yang sangat positif, dengan skor rata-rata untuk kemudahan penggunaan (5,0/5,0), kejelasan antarmuka (4,87/5,0) dan persepsi kinerja (4,95/5,0). Kesimpulannya, implementasi sistem smart scale dengan arsitektur desentralisasi terbukti merupakan solusi yang valid dan efektif untuk menjawab permasalahan latensi dan inefisiensi di lapak. Sistem ini tidak hanya unggul secara teknis dalam hal performa, tetapi juga fungsional dan dapat diterima dengan baik oleh pengguna akhir, serta berpotensi besar untuk modernisasi rantai pasok agribisnis.

Kata kunci: Smart Scale, Internet of Things, Edge Computing, Rantai Pasok Jeruk.

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan penulisan Laporan Tugas Akhir yang berjudul "Implementasi *Graph Retrieval Augmented Generation* untuk Meningkatkan Presisi Retrieval dan Validitas Sintaksis pada Konfigurasi Kubernetes". Laporan ini disusun sebagai salah satu syarat untuk menyelesaikan program Sarjana di Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Bandung, 20 Mei 2026

Penulis

Jihan Aurelia

DAFTAR ISI

LEMBAR PENGESAHAN	iii
PERNYATAAN ORISINALITAS	v
PERNYATAAN PENGGUNAAN AI	vii
ABSTRAK	ix
KATA PENGANTAR	xi
DAFTAR ISI	xiii
DAFTAR GAMBAR	xv
DAFTAR TABEL	xvii
DAFTAR PERSAMAAN	xix
DAFTAR ALGORITMA	xxi
DAFTAR <i>LISTING</i>	xxiii
DAFTAR SIMBOL	xxv
DAFTAR SINGKATAN	xxix
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	3
I.3 Tujuan	3
I.4 Batasan Masalah	4
I.5 Metodologi	5
II STUDI LITERATUR	9
II.1 Arsitektur <i>Cloud</i> Modern dan Tantangan Kompleksitas	9
II.2 Kubernetes: Platform Orkestrasi Kontainer untuk Aplikasi <i>Cloud-Native</i>	10
II.2.1 Gambaran Umum Arsitektur	10
II.2.2 <i>Resource Model</i> dan Hierarki	12
II.2.3 Manifes YAML dan <i>Infrastructure as Code</i>	14
II.3 <i>Large Language Models</i> (LLM)	14
II.3.1 Definisi dan Kapabilitas	14
II.3.2 <i>Prompt Engineering</i>	15
II.4 <i>Retrieval-Augmented Generation</i> (RAG)	15
II.4.1 Arsitektur <i>Retrieval-Augmented Generation</i>	16
II.4.2 Paradigma RAG	18
II.4.3 Relevansi RAG terhadap Dokumentasi Kubernetes API	19
II.5 <i>Knowledge Graph</i> dan GraphRAG	19
II.5.1 Definisi dan Struktur	19
II.5.2 Integrasi KG dalam RAG pada Tahap Perancangan Solusi	22
II.5.3 GraphRAG: Integrasi <i>Knowledge Graph</i> dan <i>Large Language Models</i>	23
II.6 Evaluasi Sistem Model	24
II.6.1 <i>Answer Quality</i> (AnsQ)	24
II.6.2 <i>Retrieval Quality</i> (RetQ)	25
II.6.3 <i>Reasoning Quality</i> (ReaQ)	26

III ANALISIS MASALAH	29
III.1 Analisis Kondisi Saat Ini	29
III.1.1 Pemahaman Masalah Bisnis (<i>Business Understanding</i>)	29
III.1.2 Pemahaman Data (<i>Data Understanding</i>)	30
III.2 Analisis Kebutuhan	34
III.2.1 Kebutuhan Fungsional	36
III.2.2 Kebutuhan Non Fungsional	38
III.3 Alternatif Solusi	39
III.3.1 Hybrid KG–Vector RAG	39
III.3.2 Pendekatan <i>GNN-augmented GraphRAG</i> (G-Retriever)	40
III.3.3 <i>Hybrid Neural-Symbolic</i> dengan Relational DB (Pendekatan NeuSym-RAG)	41
III.4 Analisis Pemilihan Solusi	42
III.4.1 Rubrik Evaluasi Solusi	42
III.4.2 Matriks Keputusan & Justifikasi Penilaian	43
III.4.3 Analisis Pemilihan dan Penyesuaian (Selection & Adjustment)	45
IV PERANCANGAN	47
V IMPLEMENTASI	49
VI EVALUASI	51
VI.1 Metode Evaluasi	51
VI.2 Hasil Evaluasi	51
VI.3 Pembahasan Hasil Evaluasi	51
VI.4 Evaluasi Kesalahan Penulisan yang Sering Terjadi	51
VI.4.1 Penggunaan Kata ”di mana” atau ”dimana”	51
VI.4.2 Penggunaan Kata ”sedangkan” dan ”sehingga”	51
VI.4.3 Penggunaan Istilah yang Tidak Baku	52
VI.4.4 Pemisah Desimal dan Ribuan	52
VI.4.5 Daftar atau <i>List</i>	52
VI.4.6 Penggunaan Kata ”masing-masing” dan ”setiap”	53
VII PENUTUP	55
VII.1 Kesimpulan	55
VII.2 Saran	55
Lampiran A KODE PROGRAM	61
A.1 Perangkat Lunak untuk Akuisisi Data dari Sensor Ultrasonik	61
A.2 Perangkat Lunak untuk Pengolahan Data Akuisisi dan Visualisasi Hasil Pengukuran Jarak	62
Lampiran B HASIL SURVEI LAPANGAN	63
B.1 Foto Survei Lokasi 1	63
B.2 Foto Survei Lokasi 2	63
B.3 Transkrip Wawancara dengan Narasumber	63

DAFTAR GAMBAR

I.1	Tahapan model referensi CRISP-DM (Niakšu 2015)	5
II.1	Arsitektur klaster Kubernetes yang terdiri atas <i>Control Plane</i> dan <i>Worker Nodes</i> , dihubungkan melalui <i>kube-apiserver</i> (The Kubernetes Authors 2024b)	11
II.2	Arsitektur <i>Knowledge Graph</i> (Yu dkk. 2021)	20
II.3	Model arsitektur integrasi KG dalam RAG (Knollmeyer, Caymazer, dan Grossmann 2025)	28

DAFTAR TABEL

III.1	Karakteristik Komponen Target Kubernetes	33
III.2	<i>Technical Gap Analysis</i> pada Domain Kubernetes	35
III.3	Pemetaan Kebutuhan Bisnis terhadap <i>Technical Gap</i> dan Kebutuhan Fungsional	37
III.4	Tabel harga bahan tertier	37
III.5	Kebutuhan Non-Fungsional Sistem	38
III.6	Rubrik Penilaian Kriteria Evaluasi Solusi	43
III.7	Matriks Perbandingan Solusi Berdasarkan Rubrik Evaluasi	43
VI.1	Contoh penggunaan kata ”sedangkan” dan ”sehingga”	52

DAFTAR PERSAMAAN

DAFTAR ALGORITMA

DAFTAR *LISTING*

III.1	Contoh konfigurasi YAML	32
III.2	Contoh konfigurasi YAML dengan Secret	32
A.1	Binary search code	61

DAFTAR SIMBOL

Notasi	Deskripsi	Pemakaian pertama kali
α	Koefisien bobot untuk <i>loss function</i> , tingkat signifikansi statistik	75
β	Koefisien bobot untuk <i>Binary Cross-Entropy loss</i>	75
λ	Koefisien bobot untuk CTC <i>loss</i> , parameter regularisasi	75
θ	Parameter model <i>neural network</i>	75
σ	Deviasi standar	75
μ	Rata-rata (<i>mean</i>)	75
ϵ	<i>Token blank</i> /kosong dalam CTC, konstanta kecil untuk stabilitas numerik	75
Δ	Delta (selisih/perubahan nilai)	75
π	<i>Alignment path</i> dalam CTC	75
π_t	<i>Token</i> pada <i>timestep</i> t dalam <i>alignment path</i>	76
∇ atau ∂	Operator gradien atau turunan parsial	75
$\sum$	Operator penjumlahan	75
$\prod$	Operator perkalian	75
$\int$	Operator integral	75
argmax	Argumen yang memaksimalkan fungsi	76
$\sim$	Terdistribusi menurut	75
$\rightarrow$	Menuju atau konvergen ke	75
$\mathcal{L}$	Fungsi <i>loss</i> (kerugian)	75
$\mathcal{L}_{total}$	Total <i>loss function</i>	75
$\mathcal{L}_{adversarial}$ atau $\mathcal{L}_{adv}$	<i>Adversarial loss</i>	75
$\mathcal{L}_{reconstruction}$	<i>Reconstruction loss</i>	75
$\mathcal{L}_{L1}$	L1 <i>loss</i> (<i>Mean Absolute Error</i>)	75
$\mathcal{L}_{L2}$	L2 <i>loss</i> (<i>Mean Squared Error</i>)	75
$\mathcal{L}_{CTC}$	CTC (<i>Connectionist Temporal Classification</i>) <i>loss</i>	75
$\mathcal{L}_{perceptual}$	<i>Perceptual loss</i>	75

$\mathcal{L}_{pixel}$	<i>Pixel-wise loss</i>	75
$\mathcal{L}_{BCE}$	<i>Binary Cross-Entropy loss</i>	75
$\mathcal{L}_{GAN}$	<i>GAN loss</i>	75
$\mathcal{L}_{cycle}$	<i>Cycle consistency loss</i>	75
G	<i>Generator dalam GAN</i>	75
D	<i>Discriminator dalam GAN</i>	75
$G(z)$	<i>Output dari generator dengan input z</i>	75
$D(x)$	<i>Output dari discriminator dengan input x</i>	75
$G_{A \rightarrow B}$	<i>Generator dari domain A ke B</i>	75
$G_{B \rightarrow A}$	<i>Generator dari domain B ke A</i>	75
$V(D, G)$	<i>Fungsi nilai (value function) dalam GAN</i>	75
x	<i>Sampel data asli, input citra</i>	75
y	<i>Label atau ground truth, kondisi dalam conditional GAN</i>	75
z	<i>Noise vector atau representasi laten</i>	
$\mathbb{E}$	<i>Ekspektasi atau nilai harapan</i>	75
p_{data}	<i>Distribusi data asli</i>	75
p_z	<i>Distribusi prior (noise)</i>	75
$p(y x)$	<i>Probabilitas bersyarat output y given input x</i>	75
$p_t(\pi_t x)$	<i>Probabilitas token pada timestep t</i>	
$\mathcal{B}$	<i>Fungsi penciutan (collapse function) dalam CTC</i>	75
$\mathcal{B}^{-1}$	<i>Fungsi invers penciutan dalam CTC</i>	75
T	<i>Jumlah timestep atau panjang sequence</i>	
$ x - G(y) _1$	<i>L1 distance antara ground truth dan generated image</i>	75
$ x - G(y) _2$	<i>L2 distance antara ground truth dan generated image</i>	75
$N \times N$	<i>Ukuran patch dalam PatchGAN</i>	
d	<i>Cohen's d (effect size)</i>	75
n	<i>Jumlah sampel</i>	75
p	<i>p-value (nilai probabilitas statistik)</i>	75

r	Koefisien korelasi	75
R^2	Koefisien determinasi	
$O(n)$	Notasi Big-O untuk kompleksitas linear	75
$O(n^2)$	Notasi Big-O untuk kompleksitas kuadratik	75

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
AI	<i>Artificial Intelligence</i>	1
CNN	<i>Convolutional Neural Network</i>	2
GPU	<i>Graphics Processing Unit</i>	14

BAB I

PENDAHULUAN

I.1 Latar Belakang

Transformasi industri teknologi informasi menuju arsitektur *cloud-native* kini menjadi kebutuhan fundamental, bukan sekadar tren. Gartner (2021) memproyeksikan bahwa pada tahun 2025, lebih dari 95% beban kerja digital akan di-*deploy* pada platform *cloud-native*. Angka ini meningkat tajam dari hanya 30% pada tahun 2021. Fondasi utama dari arsitektur *cloud-native* ini adalah penggunaan teknologi kontainer yang memungkinkan aplikasi dikemas dan dijalankan secara efisien. Namun, seiring dengan peningkatan skala sistem, pengelolaan ribuan kontainer secara manual tidak lagi memungkinkan sehingga dibutuhkan sebuah alat otomatisasi. Untuk menjembatani kebutuhan kompleks inilah, Kubernetes (K8s) hadir di pusat ekosistem dan mengukuhkan posisinya sebagai standar industri *de facto* untuk orkestrasi kontainer.

Berdasarkan laporan survei *Cloud Native Computing Foundation* (CNCF) tahun 2023, Kubernetes mendominasi pangsa pasar global karena ekosistemnya yang matang, dukungan komunitas yang luas, serta fleksibilitas dalam mengelola infrastruktur berskala besar secara deklaratif (Cloud Native Computing Foundation 2023). Pada tahun 2022, sekitar 58% pengguna telah menjalankan Kubernetes di lingkungan produksi dan 23% lainnya masih dalam tahap evaluasi (total 81%). Pada tahun 2023, angka tersebut meningkat menjadi 66% pengguna di produksi dengan 18% dalam evaluasi (total 84%). Peningkatan signifikan ini menunjukkan bahwa Kubernetes semakin menjadi fondasi utama dalam pengelolaan layanan *cloud-native* modern.

Meskipun Kubernetes mendominasi lanskap orkestrasi kontainer, platform ini menghadirkan tantangan operasional yang signifikan berupa kompleksitas konfigurasi berbasis berkas YAML. Paradigma deklaratif Kubernetes menuntut pengembang untuk menulis ratusan baris kode yang verbos dan hierarkis demi mendefinisikan *desired state* dari sebuah aplikasi yang mencakup berbagai spesifikasi parameter dari Kubernetes. Kompleksitas ini diperparah oleh interdependensi antar-objek yang

sangat ketat namun sering kali tidak terlihat secara eksplisit (*implicit dependencies*) yang mengakibatkan kesalahan kecil seperti ketidakcocokan *label selector* dapat memicu kegagalan sistem berantai. Akibat beban kognitif yang tinggi ini, kerumitan YAML tidak hanya menjadi hambatan skalabilitas utama bagi 65% penggunanya, tetapi juga memicu epidemi miskonfigurasi operasional dan keamanan; tercatat sekitar 80% insiden operasional berakar pada kompleksitas ini, dan 18% berkas konfigurasi sumber terbuka terbukti mengandung pelanggaran standar keamanan kritis industri (Rahman dkk. 2023).

Situasi tersebut berdampak langsung pada pemanfaatan *Large Language Models* (LLM) untuk menghasilkan kode maupun konfigurasi teknis Kubernetes. Walaupun model seperti GPT-4 menawarkan kemudahan dalam memahami dokumentasi API, kemampuan tersebut tidak selalu sejalan dengan ketepatan semantik. Studi evaluatif oleh Liu dkk. (2024) menunjukkan bahwa LLM dapat menghasilkan kode yang secara sintaksis tampak benar, tetapi keliru secara semantis. Dalam lingkungan *production-grade*, kesalahan semantik semacam ini tidak hanya bersifat minor, tetapi berpotensi memicu hilangnya layanan, gangguan orkestrasi, hingga *downtime* berskala besar.

Upaya mitigasi telah dilakukan melalui *Retrieval-Augmented Generation* (RAG) berbasis vektor untuk menambahkan konteks dokumentasi saat LLM menjawab pertanyaan teknis. Namun, penelitian oleh Wan dkk. (2025) mengungkapkan bahwa pendekatan vektor RAG hanya mengutamakan kesamaan *embedding* sehingga gagal menangkap relasi kausal, hierarkis, dan berbasis aturan teknis yang diperlukan untuk memahami konfigurasi domain Kubernetes. Pendekatan vektor RAG hanya mencapai 63,4% *exact match accuracy* dan 69,8% *context precision*, menunjukkan rendahnya ketepatan semantik dalam jawaban sistem. *Embedding* yang dilatih dari korpus umum juga mengabaikan terminologi teknis spesifik sehingga menghasilkan jawaban yang tampak relevan secara teks, tetapi tidak tepat secara domain.

Sebagai solusi, Wan dkk. (2025) mengusulkan penggabungan representasi pengetahuan terstruktur melalui *Knowledge Graph* (KG) untuk memberikan fondasi penalaran relasional yang eksplisit. Integrasi KG dalam sistem RAG menghasilkan peningkatan kinerja yang signifikan, mencapai 77,8% *exact match accuracy* dan 76,5% *context precision*, yaitu peningkatan lebih dari 14,4 poin persentase dibanding pendekatan vektor RAG. KG tidak hanya meningkatkan presisi pencarian informasi, tetapi juga memungkinkan *semantic alignment* melalui pembatasan ontologis sehingga model tidak hanya memilih teks yang mirip, tetapi juga mengakses

pengetahuan yang benar dalam domain.

Dengan demikian, pendekatan Graph-RAG menawarkan fondasi penalaran teknis yang lebih dapat dipertanggungjawabkan dan mampu menurunkan risiko halusinasi semantik berbasis domain, khususnya pada sistem *cloud-native* yang sensitif terhadap kesalahan konfigurasi. Berdasarkan urgensi tersebut, penelitian ini bertujuan untuk membangun sistem *Retrieval-Augmented Generation* (RAG) berbasis *Knowledge Graph* yang dikhususkan untuk dokumentasi Kubernetes. Pendekatan ini diharapkan mampu meningkatkan akurasi *retrieval* terhadap pertanyaan yang membutuhkan penalaran struktural dengan mengekspresikan relasi ontologis antar objek konfigurasi Kubernetes. Selain menghasilkan jawaban teknis berupa penjelasan dan konfigurasi YAML yang valid, sistem juga dirancang untuk memberikan jejak penalaran graf sebagai mekanisme pendeteksi halusinasi. Hal ini memungkinkan setiap keluaran tidak hanya relevan secara tekstual, tetapi juga dapat diverifikasi kesesuaiannya terhadap skema objek Kubernetes.

I.2 Rumusan Masalah

Penelitian ini bertujuan untuk menjawab pertanyaan-pertanyaan berikut,

1. Bagaimana membangun *knowledge graph* dari spesifikasi Kubernetes guna merepresentasikan struktur dan relasi antar-objek konfigurasi secara komprehensif?
2. Bagaimana merancang mekanisme GraphRAG yang memanfaatkan representasi struktural dalam *knowledge graph* untuk meningkatkan presisi *retrieval* pada konfigurasi Kubernetes?
3. Bagaimana perbandingan kinerja antara GraphRAG, *Vanilla* RAG, dan *Vector-based* RAG dalam meningkatkan presisi *retrieval* serta validitas sintaksis berkas konfigurasi Kubernetes?

I.3 Tujuan

Berdasarkan rumusan masalah yang telah dijelaskan pada Bab I.2, tujuan dalam penulisan penelitian ini adalah sebagai berikut,

1. Membangun *knowledge graph* dari spesifikasi OpenAPI Kubernetes (swagger.json) melalui *pipeline* ekstraksi guna memetakan entitas beserta relasi hierarkis dan referensial antar-objek konfigurasi.
2. Mengembangkan mekanisme GraphRAG dengan menelusuri jalur relasional (*graph traversal*) berdasarkan kueri pengguna, kemudian menginjeksikan *reasoning path* tersebut ke dalam *Large Language Model* (LLM) untuk menghasilkan jawaban konsisten sesuai logika domain Kubernetes.

3. Mengevaluasi kinerja metode GraphRAG menggunakan parameter *context metrics*, *faithfulness*, dan *syntactic validity*, serta membandingkannya dengan *Vanilla* RAG maupun *Vector-based* RAG dalam meningkatkan presisi *retrieval* dan validitas sintaksis keluaran YAML menggunakan model generatif GPT-4o-mini.

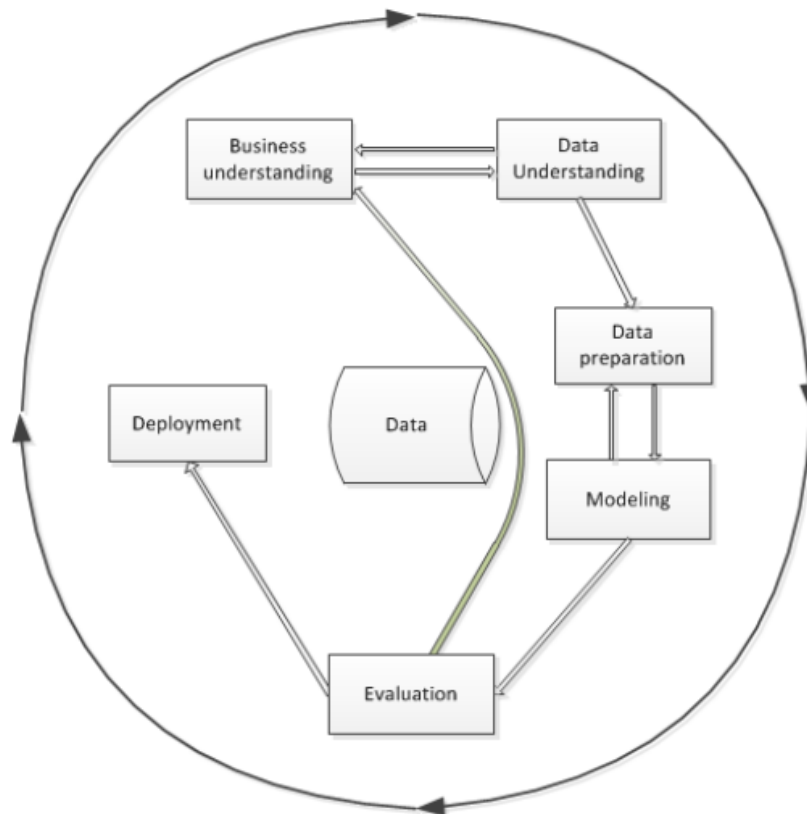
I.4 Batasan Masalah

Agar penelitian ini tetap fokus dan *feasible* dalam lingkup Tugas Akhir, batasan-batasan berikut akan diterapkan,

1. Data penelitian bersumber secara eksklusif dari spesifikasi resmi Kubernetes OpenAPI (`swagger.json`) versi v1.30 tanpa melakukan *web scraping* dokumentasi HTML. Ekstraksi data dibatasi murni pada blok *definitions* dengan mengeliminasi blok operasional protokol API (seperti *paths*, *parameters*, dan *responses*) guna mencegah *context noise* pada LLM.
2. *Knowledge graph* yang dibangun hanya merepresentasikan struktur skema (*schema structure*) dan tidak mencakup perilaku operasional (*runtime behavior*) maupun logika validasi dengan fokus utama penelitian untuk menguji validitas struktur sintaksis YAML.
3. Luaran penelitian berupa manifes Kubernetes YAML murni (*plain YAML*) berdasarkan *OpenAPI Specification*, dilengkapi jejak penalaran graf (*graph reasoning path*) guna mendeteksi halusinasi LLM.
4. Luaran file YAML tidak diuji langsung pada kluster Kubernetes nyata (*actual cluster*). Pengujian sistem murni berfokus pada validitas sintaksis menggunakan pustaka *pyyaml* serta evaluasi kinerja berdasarkan dataset uji tervalidasi pakar.
5. Model LLM yang digunakan berasal dari *pre-trained models* yang diakses melalui API (GPT-4o-mini) tanpa proses *fine-tuning*.
6. Algoritma *graph traversal* dibatasi maksimal 4 lompatan (*hops*) sesuai analisis topologi objek Kubernetes. Pembatasan ini bertujuan mengoptimalkan *Context Precision* serta efisiensi beban komputasi pada sumber daya lokal.
7. Proses pengolahan dan pra-pemrosesan data dibatasi pada lingkungan **Google Colaboratory** dan **Jupyter Notebook**.
8. Kinerja komputasi dan hasil eksperimen didasarkan pada perangkat keras spesifik (PC dengan CPU AMD Ryzen 5, GPU AMD Radeon RX 6750XT, RAM 16 GB) dan mungkin bervariasi pada konfigurasi lain.

I.5 Metodologi

Pelaksanaan Tugas Akhir ini mengadopsi kerangka kerja *Cross-Industry Standard Process for Data Mining* (CRISP-DM) yang dapat dilihat pada Gambar I.1 sebagai metodologi utama. CRISP-DM dipilih karena memberikan proses terstruktur, iteratif, dan dapat diadaptasi untuk pengembangan sistem berbasis analisis data dan representasi pengetahuan (Niakšu 2015). Tahapan metodologi yang digunakan terdiri dari enam fase berikut,



Gambar I.1 Tahapan model referensi CRISP-DM (Niakšu 2015)

1. Pemahaman Bisnis (*Business Understanding*)

Tahap ini bertujuan untuk mengidentifikasi kebutuhan serta masalah inti yang ingin diselesaikan melalui pemanfaatan data. Dalam konteks penelitian ini, permasalahan yang diangkat adalah ketidaktepatan jawaban teknis yang dihasilkan *Large Language Models* (LLM) ketika merujuk dokumentasi Kubernetes, terutama pada konfigurasi YAML yang bersifat struktural dan rentan mengandung halusinasi. Oleh karena itu, kebutuhan penelitian diarahkan pada peningkatan akurasi penalaran struktural pada proses *retrieval* serta mitigasi halusinasi konfigurasi pada sistem *cloud-native*.

2. Pemahaman Data (*Data Understanding*)

Fase ini berfokus pada pengumpulan dan eksplorasi sumber data untuk mema-

hami struktur, kualitas, serta batasannya. Pada penelitian ini, data diperoleh dari spesifikasi resmi Kubernetes OpenAPI yang dianalisis untuk memahami jenis entitas konfigurasi, atribut yang dimiliki, serta relasi hierarkis dan referensial antar objek. Pemahaman ini menentukan objek Kubernetes yang akan dimodelkan dalam *Knowledge Graph*.

3. **Persiapan Data (*Data Preparation*)**

Tahapan ini mencakup pembersihan dan transformasi data agar siap digunakan dalam proses pemodelan. Dalam penelitian ini, dilakukan ekstraksi otomatis dari spesifikasi OpenAPI Kubernetes (JSON/YAML) untuk membentuk representasi entitas, yaitu relasi yang dapat dimodelkan sebagai *Knowledge Graph*. Proses ini mencakup identifikasi atribut, pemetaan hierarki *parent-child*, serta penghubungan *cross-resource references*. Hasil akhirnya berupa struktur graf yang terstandarisasi dan siap digunakan pada mekanisme *retrieval*.

4. **Pemodelan (*Modeling*)**

Fase ini bertujuan untuk menerapkan algoritma yang sesuai guna menyelesaikan masalah yang telah dirumuskan. Pada penelitian ini, dibangun mekanisme *graph-guided retrieval* yang menelusuri graf menggunakan *graph traversal* berdasarkan kueri pengguna dan menghasilkan *reasoning path*. Jalur penalaran tersebut kemudian diinjeksikan sebagai konteks pada proses RAG untuk memandu LLM dalam menghasilkan jawaban serta konfigurasi YAML yang sesuai dengan logika sistem Kubernetes.

5. **Evaluasi (*Evaluation*)**

Tahap evaluasi bertujuan menilai kualitas model dan membandingkannya dengan pendekatan alternatif menggunakan metrik terukur. Dalam penelitian ini, kinerja sistem dievaluasi menggunakan metrik *context precision*, *context recall*, *faithfulness*, dan *Syntactic Validity*, kemudian dibandingkan dengan dua *baseline*, yaitu *Vanilla LLM* dan *Vector-based RAG*. Evaluasi dilakukan untuk membuktikan peningkatan relevansi konteks dan penurunan halusinasi semantik pada jawaban teknis.

6. **Penyebaran (*Deployment*)**

Fase ini berkaitan dengan penyajian hasil implementasi dalam bentuk prototipe dan dokumentasi. Pada penelitian ini, penyebaran diwujudkan dalam bentuk antarmuka interaktif sederhana berupa CLI yang menampilkan jawaban, konfigurasi YAML, serta *reasoning path* sebagai bukti penalaran graf. Selain itu, dokumentasi lengkap disusun sebagai bentuk penyebaran akademik yang memuat rancangan sistem, hasil evaluasi, serta rekomendasi penelitian

selanjutnya.

BAB II

STUDI LITERATUR

II.1 Arsitektur *Cloud* Modern dan Tantangan Kompleksitas

Pergeseran paradigma dari arsitektur monolitik menuju layanan mikro (*microservices*) kini telah menjadi fondasi utama dalam pengembangan aplikasi *cloud-native* modern. Berbeda dengan pendekatan arsitektur tradisional, gaya arsitektur layanan mikro mendistribusikan logika bisnis secara masif ke dalam puluhan hingga ratusan layanan kecil independen. Distribusi fungsionalitas ini memunculkan kebutuhan akan mekanisme penerapan (*deployment*) mandiri. Sebagai solusi, layanan mikro secara alami terintegrasi dengan platform berbasis kontainer karena setiap layanan dapat dikemas dan didistribusikan di dalam kontainernya masing-masing (Soldani, Tamburri, dan Van Den Heuvel 2018). Walaupun memberikan keuntungan operasional yang signifikan berupa portabilitas lintas *cloud*, skalabilitas tingkat tinggi, dan kebebasan pemilihan teknologi bagi pengembang, ekosistem kontainer ini memunculkan paradoks kompleksitas baru (Soldani, Tamburri, dan Van Den Heuvel 2018).

Dalam lingkungan terdistribusi yang melibatkan banyak kontainer ini, orkestrasi penerapan layanan mikro menjadi sebuah tantangan tersendiri. Setiap layanan mikro di dalam kontainer berkomunikasi melalui mekanisme antarmuka pemrograman aplikasi (API) seperti protokol HTTP. Spesifikasi API beserta konfigurasi orkestrasinya bertindak sebagai kontrak komunikasi esensial antar-layanan (Soldani, Tamburri, dan Van Den Heuvel 2018). Kontrak struktural ini sama sekali tidak boleh dilanggar guna menjaga stabilitas operasional serta memastikan layanan-layanan tersebut dapat terus saling berinteraksi secara dinamis (Soldani, Tamburri, dan Van Den Heuvel 2018). Ketergantungan yang sangat ketat di bawah kendali sistem orkestrasi ini menuntut pemahaman komprehensif terhadap interaksi antar-layanan sebagai kunci utama keberhasilan operasional aplikasi *cloud*.

Akibatnya, dokumentasi API dan manifes konfigurasi orkestrasi mengalami lonjakan kompleksitas. Sebagaimana diidentifikasi oleh Soldani, Tamburri, dan Van Den Heuvel (2018), pemahaman kognitif yang keliru terhadap arsitektur dan interaksi

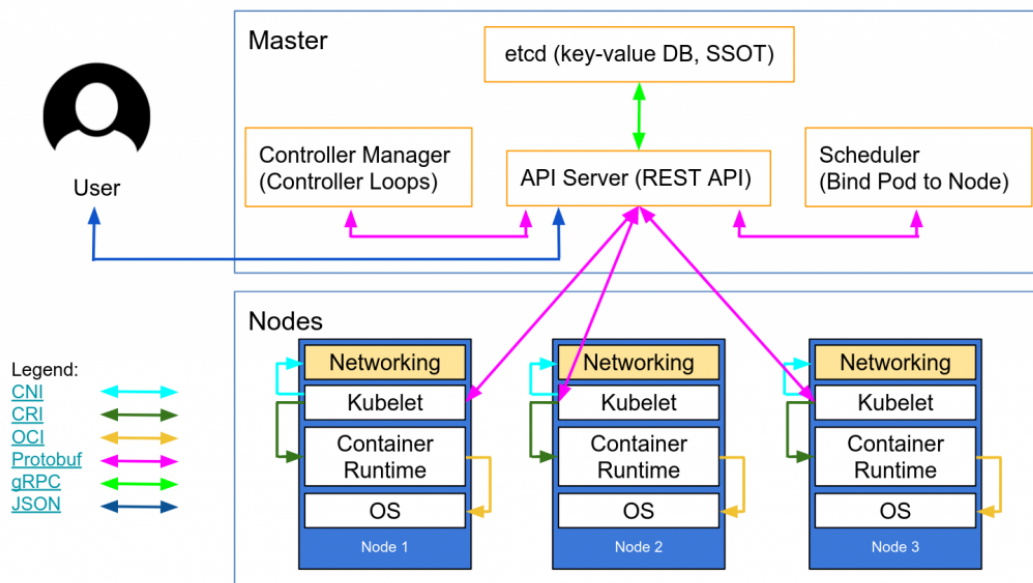
kontrak API ini sering kali berujung pada kegagalan operasional. Jika tidak diisolasi dengan tepat dalam sistem orkestrasi, kegagalan komunikasi pada satu layanan mikro dapat memicu rentetan kegagalan pada layanan lain yang bergantung padanya. Kondisi ini pada akhirnya menghasilkan fenomena kegagalan sistem berantai (*cascading failures*) yang melumpuhkan seluruh aplikasi (Soldani, Tamburri, dan Van Den Heuvel 2018).

II.2 Kubernetes: Platform Orkestrasi Kontainer untuk Aplikasi *Cloud-Native*

Kubernetes merupakan platform orkestrasi kontainer yang dirancang untuk mengotomatisasi proses *deployment*, *scaling*, dan manajemen aplikasi berbasis kontainer dalam lingkungan *cloud-native* (The Kubernetes Authors 2024b). Platform ini menyediakan mekanisme deklaratif berbasis konfigurasi yang memungkinkan pengguna mendefinisikan keadaan sistem yang diinginkan (*desired state*) melalui berkas konfigurasi, sementara Kubernetes menjaga agar keadaan tersebut tetap terpenuhi melalui rangkaian komponen pengendali (*controller loops*).

II.2.1 Gambaran Umum Arsitektur

Arsitektur Kubernetes dibangun atas konsep klaster yang terdiri dari dua kelompok komponen utama, yaitu *Control Plane* dan *Worker Nodes*. *Control Plane* berfungsi sebagai pengendali utama yang membuat keputusan bisnis aplikasi, misalnya penjadwalan, pemantauan, serta konsistensi status. *Worker Nodes* merupakan entitas komputasi fisik/virtual yang menjalankan kontainer sebagai satuan aplikasi. Kedua komponen ini berkomunikasi secara internal melalui API server yang menjadi pintu utama seluruh permintaan sistem.



Gambar II.1 Arsitektur kluster Kubernetes yang terdiri atas *Control Plane* dan *Worker Nodes*, dihubungkan melalui *kube-apiserver* (The Kubernetes Authors 2024b)

1. *Control Plane*

Control Plane bertanggung jawab mengelola keadaan (*state*) kluster secara keseluruhan. Komponen utamanya meliputi,

- kube-apiserver** sebagai gerbang akses seluruh operasi manipulasi objek konfigurasi melalui API,
- etcd** sebagai basis data *key-value* yang menyimpan informasi konfigurasi dan status,
- kube-scheduler** yang menentukan Node penempatan Pod berdasarkan ketersediaan sumber daya,
- kube-controller-manager** yang berisi sekumpulan kontroler untuk menjaga kesesuaian antara kondisi aktual dan *desired state*. Melalui mekanisme deklaratif ini, Kubernetes secara otomatis memperbaiki status objek yang tidak sesuai dan memastikan setiap aplikasi berjalan sesuai aturan konfigurasi yang ditentukan.

2. *Worker Nodes*

Worker Nodes merupakan lapisan eksekusi tempat aplikasi berjalan dalam bentuk Pod. Node terdiri atas tiga komponen inti:

- kubelet** yang bertugas mengeksekusi Pod sesuai *specification* yang diterima dari *Control Plane*,
- kube-proxy** yang menyediakan layanan *network forwarding* dan *load balancing* sederhana antar Pod dalam kluster,

- c. **container runtime** yang bertanggung jawab menjalankan kontainer. Dengan demikian, Node menjalankan komputasi terdistribusi yang dikontrol oleh *Control Plane* melalui komunikasi berbasis API.

II.2.2 Resource Model dan Hierarki

Dalam *resource model* Kubernetes, terdapat beberapa objek inti pembangun aplikasi yang paling sering digunakan:

- **Pod**: Unit komputasi terkecil sebagai pembungkus kontainer aplikasi
- **Deployment**: Pengatur replika dan proses pembaruan versi untuk mencapai ketersediaan tinggi
- **Service**: Penyedia titik akses jaringan yang stabil untuk aplikasi di dalam *Pod*
- **Ingress**: Pengatur rute lalu lintas eksternal masuk ke dalam kluster
- **ConfigMap**: Penyimpan variabel lingkungan dan konfigurasi non-sensitif
- **Secret**: Pengelola data sensitif seperti kredensial dan token
- **PersistentVolumeClaim**: Permintaan penyimpanan persisten untuk data yang harus bertahan
- **ServiceAccount**: Identitas yang digunakan oleh *Pod* untuk berinteraksi dengan API server
- **Role** dan **RoleBinding**: Kontrol akses berbasis peran (RBAC)
- **HorizontalPodAutoscaler**: Pengatur skala otomatis berdasarkan metrik penggunaan

Antar-objek ini saling terikat melalui berbagai jenis relasi yang dapat dikategorikan sebagai berikut:

1. Relasi Struktural (*Structural Relationships*)

- **HAS_PROPERTY**: Hubungan antara objek induk dengan properti skema yang dimilikinya (misal: *Deployment* memiliki properti *spec*)
- **EXTENDS**: Hubungan pewarisan skema dari mekanisme *allOf* (misal: *Deployment* extends *ObjectMeta*)
- **ONE_OF**: Hubungan tipe union yang mengharuskan pemilihan satu dari beberapa opsi (misal: *Volume* memilih satu sumber penyimpanan)
- **ANY_OF**: Hubungan tipe alternatif yang memungkinkan satu atau lebih opsi (misal: *EnvFromSource* dapat menggunakan *ConfigMap* atau *Secret*)

2. Relasi Workload (*Workload Relationships*)

- **CONTAINS_POD_TEMPLATE**: Hubungan antara controller (*Deployment*, *ReplicaSet*, *DaemonSet*, *StatefulSet*, *Job*) dengan template *Pod* yang dikelolanya

- CONTAINS_JOB_TEMPLATE: Hubungan antara *CronJob* dengan template *Job* yang dijadwalkan
- HAS_CONTAINER: Hubungan antara *PodSpec* dengan definisi *Container* yang dijalankan di dalamnya

3. Relasi Penyimpanan (*Storage Relationships*)

- CLAIMS_VOLUME: Hubungan antara *StatefulSet* dengan *PersistentVolumeClaim* untuk penyimpanan persisten
- MOUNTS_VOLUME: Hubungan antara *PodSpec* dengan *Volume* yang di-mount ke kontainer
- USES_STORAGE_CLASS: Hubungan antara *PersistentVolumeClaim* dengan *StorageClass* yang menentukan jenis penyimpanan

4. Relasi Konfigurasi (*Configuration Relationships*)

- LOADS_CONFIGMAP: Hubungan antara *Container* dengan *ConfigMap* yang digunakan untuk injeksi konfigurasi
- USES_SECRET: Hubungan antara *PodSpec* dengan *Secret* untuk data sensitif

5. Relasi Jaringan (*Networking Relationships*)

- SELECTS_POD: Hubungan antara *Service* dengan *Pod* yang dipilih melalui label selector
- ROUTES_TO_SERVICE: Hubungan antara *Ingress* dengan *Service* tujuan rute lalu lintas

6. Relasi RBAC (*Role-Based Access Control Relationships*)

- BINDS_ROLE: Hubungan antara *RoleBinding/ClusterRoleBinding* dengan *Role/ClusterRole* yang diikat
- BINDS_SERVICE_ACCOUNT: Hubungan antara *RoleBinding/ClusterRoleBinding* dengan *ServiceAccount* yang diberi izin
- USES_SERVICE_ACCOUNT: Hubungan antara *PodSpec* dengan *ServiceAccount* yang digunakan untuk identitas

7. Relasi Autoscaling (*Autoscaling Relationships*)

- SCALES_RESOURCE: Hubungan antara *HorizontalPodAutoscaler* dengan workload target (*Deployment, StatefulSet, ReplicaSet*) yang diskalakan

Pemetaan relasi kepemilikan (*ownership*) dan ketergantungan fungsional (*dependencies*) inilah yang akan secara langsung dikonversi menjadi skema graf pengetahuan (*knowledge graph*) pada penelitian ini. Sebagai contoh, sebuah *Deployment* secara hierarkis memiliki sub-sumber daya *PodSpec* melalui relasi CONTAINS_JOB_TEMPLATE, dan *Pod* tersebut memiliki ketergantungan operasional terhadap *ConfigMap* tertentu melalui relasi LOADS_CONFIGMAP. Total terdapat 21 jenis edge yang terdistribusi

dalam 7 kategori relasi, membentuk ontologi lengkap untuk generasi YAML Kubernetes.

II.2.3 Manifes YAML dan *Infrastructure as Code*

Pengembang berinteraksi dengan model sumber daya Kubernetes melalui pendekatan *Infrastructure as Code* (IaC) (The Kubernetes Authors 2024a). Pendekatan ini mewajibkan pengguna untuk mendeklarasikan wujud infrastruktur yang diinginkan (*desired state*) ke dalam bentuk dokumen teks berformat YAML. Setiap manifes YAML standar wajib memiliki empat atribut struktural utama agar dapat diterima oleh API Kubernetes:

1. `apiVersion`: Menentukan versi skema API pembungkus objek.
2. `kind`: Mendefinisikan jenis entitas sumber daya.
3. `metadata`: Memuat data identifikasi unik seperti nama, label pengelompokan, dan *namespace*.
4. `spec`: Berisi spesifikasi teknis rinci penentu sifat dan bentuk hierarki objek tersebut.

Penyusunan manifes YAML menuntut pemahaman terkait batasan skema. Pengembang harus bisa membedakan secara presisi antara ruas wajib (*required*) penentu validitas dan ruas opsional (*optional*) penambah fitur khusus. Praktik terbaik (*best practices*) dalam penulisan IaC menyarankan modularitas dokumen, konsistensi penggunaan label, dan pemisahan logika aplikasi dari data konfigurasi. Sebuah struktur manifes YAML yang mematuhi kelengkapan ruas wajib (*field requirements*) dan valid secara sintaksis inilah yang menjadi representasi target luaran yang akan di-bangkitkan oleh sistem generasi pada penelitian ini.

II.3 *Large Language Models* (LLM)

II.3.1 Definisi dan Kapabilitas

Large Language Model (LLM) merupakan model kecerdasan buatan berbasis jaringan saraf berskala besar yang dilatih pada korpus teks *multi-domain* untuk mempelajari representasi bahasa alami secara mendalam (Wan dkk. 2025). Melalui proses pelatihan tersebut, LLM mampu menginterpretasikan konteks linguistik yang kompleks serta menghasilkan keluaran berupa teks yang koheren, kontekstual, dan adaptif terhadap berbagai jenis tugas seperti *question answering*, penalaran, serta peringkasan teks.

Meskipun unggul dalam pemahaman bahasa alami, LLM memiliki keterbatasan faktual yang signifikan ketika diterapkan pada domain teknis, terutama terhadap pengetahuan yang presisi, bersifat prosedural, serta memerlukan landasan ontologis.

Beberapa keterbatasan yang disorot oleh Wan dkk. (2025) meliputi,

- a. *Hallucination*, yaitu kemampuan menghasilkan jawaban yang secara linguistik meyakinkan tetapi tidak memiliki landasan fakta yang jelas sehingga berisiko menyesatkan proses pengambilan keputusan teknis.
- b. Keterbatasan cakupan pengetahuan, LLM hanya mengandalkan data pelatihan awal sehingga tidak selalu mencakup informasi domain yang mutakhir.
- c. Ketiadaan atribusi sumber, yaitu LLM tidak dapat memberikan justifikasi atau referensi eksplisit terhadap jawaban yang dihasilkan.

Keterbatasan tersebut menjadi alasan utama perlunya integrasi antara LLM dan arsitektur *retrieval-augmented generation* (RAG).

II.3.2 *Prompt Engineering*

Prompt engineering merupakan teknik perancangan instruksi masukan bagi LLM untuk mengendalikan keluaran model berdasarkan tujuan tugas, cakupan konteks, serta batasan informasi yang diizinkan (Wan dkk. 2025). Dalam domain teknis, teknik ini tidak hanya berperan sebagai instruksi linguistik, tetapi juga sebagai representasi pengetahuan eksplisit yang mendefinisikan bagaimana LLM harus memanfaatkan sumber informasi domain.

Efektivitas keluaran LLM dipengaruhi secara signifikan oleh strategi *prompt engineering* berikut,

- a. Spesifikasi tugas yang eksplisit, termasuk definisi peran model dan batasan prosedural, misalnya instruksi untuk membatasi jawaban pada proses manufaktur tertentu.
- b. Penyediaan konteks terstruktur, melalui penyisipan entitas domain, hubungan semantik, serta cuplikan dokumen terkait yang relevan dengan pertanyaan.
- c. Pemberian batasan (*constraint injection*) yang secara eksplisit membatasi ruang solusi berdasarkan pengetahuan terbatas dalam dokumen yang valid.
- d. Penyusunan format jawaban, misalnya dalam bentuk tabel, daftar berhierarki, atau penjelasan berbasis pembuktian prosedural.

Strategi tersebut tidak hanya mengontrol keluaran, tetapi juga mencegah LLM menafsirkan konteks di luar pengetahuan domain sehingga berperan sebagai mekanisme mitigasi terhadap fenomena *hallucination*.

II.4 *Retrieval-Augmented Generation* (RAG)

Retrieval-Augmented Generation (RAG) merupakan arsitektur pemrosesan bahasa alami yang mengombinasikan mekanisme *Information Retrieval* (IR) dengan kemampuan generatif *Large Language Models* (LLM). Pendekatan ini dikembangkan

untuk mengatasi keterbatasan memori parametris pada model bahasa, khususnya dalam menangani tugas yang bersifat *knowledge-intensive*, yaitu model rentan menghasilkan informasi yang keliru atau tidak berbasis fakta (*hallucination*) (Antonio Moreno-Cediel 2025).

Berbeda dengan metode generatif konvensional yang hanya mengandalkan pengetahuan tersimpan selama proses pelatihan, RAG memanfaatkan basis pengetahuan non-parametris melalui proses pencarian (*retrieval*) sehingga jawaban yang dihasilkan lebih akurat dan relevan terhadap konteks masukan.

II.4.1 **Arsitektur *Retrieval-Augmented Generation***

Arsitektur *Retrieval-Augmented Generation* (RAG) merupakan pendekatan hibrida yang menggabungkan kemampuan penalaran *Large Language Models* (LLM) dengan basis pengetahuan eksternal melalui proses pencarian semantik. Struktur ini dirancang untuk mengurangi *hallucination*, meningkatkan akurasi fakta, serta mendukung pembaruan pengetahuan secara dinamis (Gao dkk. 2024). Secara umum, alur kerja RAG terdiri atas empat tahapan teknis utama yang berurutan dan saling bergantung, yaitu *Data Ingestion & Indexing*, *Retrieval*, *Augmentation*, dan *Generation*.

1. ***Data Ingestion & Indexing***

Tahapan ini bersifat *offline* karena dilakukan sebelum sistem menjawab pertanyaan. Tujuan utamanya adalah mentransformasi data mentah menjadi basis pengetahuan terstruktur yang dapat dengan mudah dicari oleh mesin. Prosesnya terdiri dari langkah-langkah berikut,

a. ***Akuisisi Data***

Sistem mengumpulkan data dari berbagai sumber seperti dokumentasi teknis, *API specification*, artikel ilmiah, *knowledge base*, atau korpus web.

b. ***Preprocessing***

Pembersihan dokumen dilakukan melalui normalisasi format, penghapusan *noise*, serta ekstraksi metadata sehingga struktur informasi menjadi homogen dan konsisten.

c. ***Chunking***

Dokumen panjang dipecah menjadi unit informasi kecil (*chunks*) berukuran 200–500 token agar sesuai dengan batas konteks LLM dan dapat di-*retrieve* secara efisien.

d. ***Embedding***

Setiap *chunk* diubah menjadi representasi numerik berbasis vektor meng-

gunakan *sentence embedding models* seperti MiniLM, BERT, atau struktur densitas modern lainnya.

e. *Indexing*

Hasil *embedding* disimpan pada *vector store* (misalnya FAISS, Chroma, Weaviate, Pinecone) sehingga pencarian dapat dilakukan berdasarkan kedekatan semantik antar vektor.

2. *Retrieval*

Proses *retrieval* dilakukan ketika pengguna mengajukan pertanyaan. Sistem akan mencari konteks paling relevan melalui mekanisme berikut,

a. *Embedding* Pertanyaan

Pertanyaan pengguna diubah menjadi vektor menggunakan model *embedding* yang sama dengan proses indexing.

b. *Similarity Search*

Dilakukan pencocokan vektor menggunakan metrik jarak seperti *cosine similarity*, *dot-product*, atau *Euclidean distance*.

c. *Top-k Selection*

Sistem memilih sejumlah dokumen teratas (umumnya $k = 3$ atau $k = 5$) yang memiliki skor kemiripan tertinggi dan mengandung konteks yang berpotensi menjawab pertanyaan.

Kualitas *retrieval* yang baik menjadi faktor kunci dalam menurunkan risiko *hallucination* pada LLM karena jawaban dibatasi oleh fakta yang relevan.

3. *Augmentation*

Tahap *augmentation* bertugas menyisipkan konteks hasil *retrieval* ke dalam *prompt* secara sistematis sehingga dapat dipahami oleh LLM. Prosesnya mencakup tahapan berikut,

a. *Concatenation*

Sistem menggabungkan pertanyaan pengguna dengan konten dokumen yang ditemukan.

b. *Formatting*

Struktur informasi dapat diberikan dalam format teks baku, tabel, *JSON schema*, *instruction-style*, atau bentuk terstruktur lainnya.

c. *Constraint Injection*

Sistem memberikan batasan eksplisit, misalnya “jawab hanya menggunakan konteks yang diberikan, jangan menambah informasi baru”, untuk mencegah LLM berhalusinasi.

II.4.2 Paradigma RAG

Retrieval-Augmented Generation berkembang dalam tiga paradigma utama, yaitu *Naive RAG*, *Advanced RAG*, dan *Modular RAG*. Ketiga paradigma ini mencerminkan evolusi dari *pipeline* sederhana menuju arsitektur yang dapat diadaptasi untuk berbagai kebutuhan, mulai dari peningkatan akurasi sampai fleksibilitas sistem (Gao dkk. 2024).

1. *Naive RAG*

Paradigma ini merupakan bentuk paling dasar dari RAG. *Pipeline* umumnya terdiri dari tiga langkah, yaitu *indexing*, *retrieval*, dan *generation*. Dokumen melalui proses *chunking*, kemudian direpresentasikan menggunakan *embedding* dan disimpan pada *vector store*. Ketika pertanyaan diajukan, sistem mengambil k dokumen paling relevan menggunakan pencarian berbasis kesamaan semantik, lalu model generatif menyusun jawaban berdasarkan konteks tersebut. Keterbatasan utama pendekatan ini adalah ketergantungan penuh terhadap kualitas *chunk* dan hasil *retrieval* yang dapat mengarah pada jawaban yang tidak akurat.

2. *Advanced RAG*

Paradigma ini melakukan optimasi pada dua fase utama, yaitu *pre-retrieval* dan *post-retrieval*. Tahap *pre-retrieval* meliputi peningkatan kualitas indeks seperti pengayaan metadata, peningkatan granularitas *chunk*, serta teknik *query rewriting* dan *query expansion* untuk meningkatkan relevansi hasil pencarian. Sementara itu, *post-retrieval* dilakukan melalui *reranking* serta *context compression* agar konteks yang terkirim ke model lebih berfokus pada informasi penting saja. Paradigma ini masih mengikuti alur linear tetapi menghasilkan akurasi yang lebih tinggi dibanding *Naive RAG*.

3. *Modular RAG*

Paradigma ini memperluas arsitektur RAG menjadi sistem yang dapat dikustomisasi melalui penambahan atau penggantian modul. Contoh dari modul yang ada yaitu *Search Module* (untuk menggabungkan pencarian dari *search engine*, database, maupun *knowledge graph*), *Memory Module* (yang memanfaatkan memori internal LLM), *Routing Module* (yang memilih jalur eksekusi berdasarkan tipe pertanyaan), serta *Task Adapter Module* (yang menyesuaikan proses dengan kebutuhan tugas tertentu). Modular RAG juga memungkinkan pola *iterative retrieval* dan *adaptive retrieval* sehingga menjadikannya paradigma paling fleksibel untuk aplikasi domain spesifik.

II.4.3 Relevansi RAG terhadap Dokumentasi Kubernetes API

Dokumentasi Web API, termasuk Kubernetes, memiliki karakteristik unik berupa kombinasi deskripsi sintaks terstruktur (misalnya skema JSON/YAML) dan penjelasan semantik dalam bentuk teks alami. Penelitian Kotstein dan Decker (2024) menunjukkan bahwa pemrosesan semantik terhadap dokumentasi API dapat dilakukan tanpa ontologi formal dengan memetakan makna berdasarkan nama parameter, struktur hierarkis, dan deskripsi bahasa alami.

Dalam konteks Kubernetes, komponen API seperti Pod, Service, dan Deployment direpresentasikan melalui skema bertingkat yang bersifat kompleks dan memiliki ketergantungan semantik. Oleh karena itu, implementasi RAG pada domain ini membutuhkan,

1. ***Schema-aware semantic chunking***

Teknik ini memastikan bahwa pemotongan dokumen (*chunking*) dilakukan mengikuti struktur skema objek API. Setiap potongan teks harus mewakili satu entitas atau blok atribut yang utuh sehingga relasi antar-*field* tidak terpisah atau kehilangan konteks. Dengan demikian, representasi vektor yang dihasilkan tetap mempertahankan hierarki objek.

2. ***Path-based serialization***

Teknik ini menyimpan setiap elemen API beserta jalur lengkapnya dalam struktur objek sehingga posisi semantiknya tetap terpelihara.

3. ***Hybrid index retrieval***

Pendekatan ini mengombinasikan pencarian berbasis kesamaan semantik teks dengan pembobotan struktur sintaksis dari objek API. Selain memanfaatkan *embedding* untuk menemukan kemiripan makna antar-teks, sistem juga memberikan prioritas pada elemen yang memiliki signifikansi struktural (misalnya *field* wajib atau relasi antar-objek). Hasilnya, mekanisme *retrieval* menghasilkan konteks yang tidak hanya relevan secara linguistik, tetapi juga benar secara hierarki dan fungsi API.

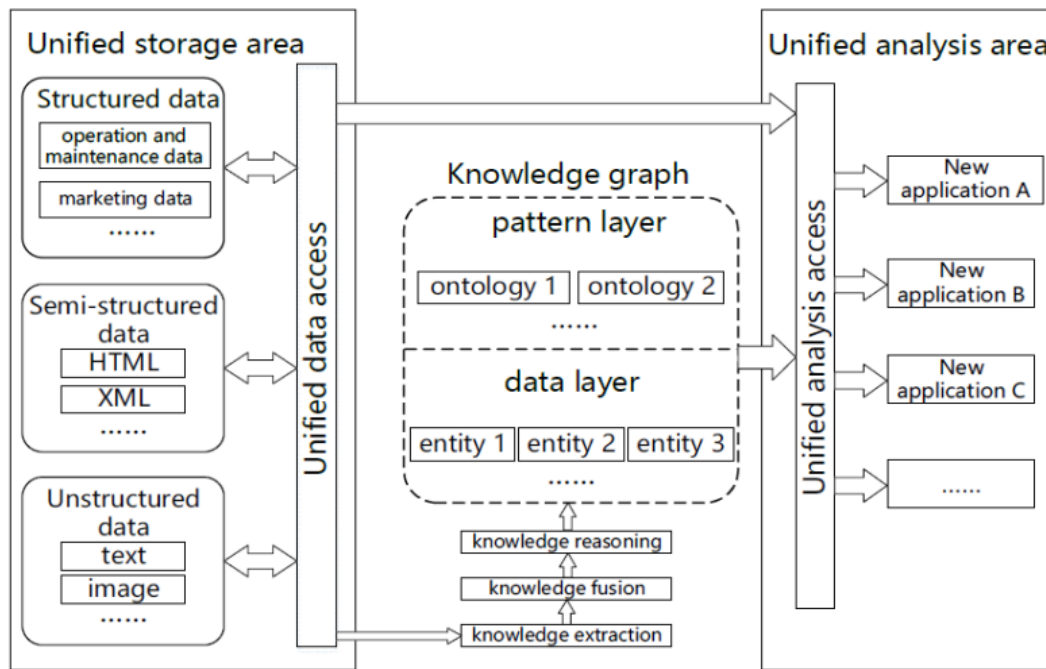
Dengan demikian, penerapan RAG pada dokumentasi Kubernetes tidak hanya berfungsi sebagai mesin pencarian semantik, tetapi juga menjadi pendekatan mitigasi *hallucination*.

II.5 Knowledge Graph dan GraphRAG

II.5.1 Definisi dan Struktur

Knowledge Graph (KG) merupakan representasi pengetahuan dalam bentuk graf yang tersusun dari entitas (*node*) dan relasi (*edge*) yang memiliki makna semantik

(Hofer dkk. 2024). Berbeda dengan basis data konvensional yang mengandalkan skema statis, KG bersifat *schema-flexible* sehingga mampu mengintegrasikan data heterogen dari sumber terstruktur, semi-terstruktur, maupun tidak terstruktur seperti yang ditunjukkan pada Gambar II.2. Struktur KG umumnya dinyatakan melalui *triples* berbentuk (subjek–predikat–objek).



Gambar II.2 Arsitektur *Knowledge Graph* (Yu dkk. 2021)

Menurut Yu dkk. (2021), terdapat dua pendekatan utama dalam pembangunan KG, yaitu *top-down* dan *bottom-up* yang dibedakan berdasarkan urutan perancangan model dan pengisian fakta.

1. *Top-down Construction*

Pada pendekatan ini, proses pembangunan KG dimulai dari definisi model semantik terlebih dahulu. Langkah pertama mencakup perancangan *ontology library* yang memuat kelas entitas, relasi, hierarki, dan aturan semantik. Setelah struktur konseptual ditetapkan, barulah fakta atau data dimasukkan ke dalam basis pengetahuan. Dengan demikian, pendekatan *top-down* mengikuti urutan *model layer* kemudian *data layer*. Pendekatan ini umumnya digunakan pada domain yang membutuhkan konsistensi semantik tinggi seperti medis, keuangan, atau layanan publik.

2. *Bottom-up Construction*

Berbeda dari pendekatan sebelumnya, *bottom-up* mengawali proses konstruksi melalui ekstraksi data langsung dari korpus teks atau sumber data lain. De-

ngan menganalisis frekuensi entitas, dan pola semantik kemudian dibentuk secara bertahap. Artinya, *data layer* dibangun terlebih dahulu, kemudian dilanjutkan dengan penyusunan *pattern layer*. Pendekatan ini didorong oleh ketersediaan data dan sesuai untuk domain yang cepat berubah atau belum memiliki referensi ontologi.

Menurut Hofer dkk. (2024), konstruksi *Knowledge Graph* (KG) modern umumnya dilakukan secara semi-otomatis dan bersifat inkremental. Proses pembangunan KG mencakup sejumlah tugas inti yang dapat dijalankan secara paralel, asinkron, maupun selektif sesuai jenis sumber data dan kebutuhan penggunaan. Tugas-tugas utama tersebut adalah sebagai berikut,

1. **Manajemen Metadata**

Pengelolaan metadata terkait, seperti *provenance* entitas, informasi temporal, struktur data, kualitas data, hingga log proses. Tugas ini bersifat menyeluruh karena diperlukan pada setiap tahapan konstruksi KG.

2. ***Data Acquisition dan Preprocessing***

Pemilihan sumber data yang relevan, akuisisi data, transformasi format, serta pembersihan awal. Tingkat detail proses ini bergantung pada keragaman sumber yang digunakan.

3. **Manajemen Ontologi**

Pembuatan dan pengembangan ontologi secara bertahap untuk mendefinisikan kelas, properti, dan relasi yang membentuk struktur semantik KG.

4. ***Knowledge Extraction (KE)***

Ekstraksi informasi terstruktur dari data tidak terstruktur atau semi-terstruktur menggunakan teknik seperti *Named Entity Recognition*, *entity linking*, dan *relation extraction*.

5. ***Entity Resolution (ER) dan Fusion***

Penyatuan entitas yang merepresentasikan objek yang sama untuk menghindari duplikasi.

6. ***Knowledge Completion***

Pelengkapan KG dengan pengetahuan baru melalui inferensi, prediksi relasi hilang, atau penambahan tipe entitas untuk meningkatkan kelengkapan semantik.

7. ***Quality Assurance (QA)***

Identifikasi masalah kualitas data pada KG serta penerapan strategi perbaikan yang dapat mencakup *validation checks*, konsistensi semantik, atau pembersihan lanjutan.

Proses ini tidak selalu bersifat linear karena urutan eksekusi bergantung pada jenis

sumber data dan beberapa langkah dapat menghilangkan kebutuhan langkah lain, seperti *entity linking* yang dapat menggantikan *entity resolution*. Selain itu, *metadata management* bersifat *cross-cutting* karena berlangsung pada seluruh tahap konstruksi KG.

II.5.2 Integrasi KG dalam RAG pada Tahap Perancangan Solusi

Sistem menggabungkan *Knowledge Graph* (KG) dengan alur RAG yang bertujuan untuk meningkatkan akurasi pencarian, kemampuan *multi-hop reasoning*, serta menjaga keterkaitan struktur dokumen. Arsitektur keseluruhan mengikuti tiga fase utama, yaitu *Indexing*, *Retrieval*, dan *Generation* (Knollmeyer, Caymazer, dan Grossmann 2025). Model arsitektur ditunjukkan pada Gambar II.3.

1. Konstruksi KG dan Ekstraksi Struktur Dokumen (*Indexing*)

Proses awal dimulai dari ekstraksi teks serta identifikasi elemen struktur dokumen. Proses *chunking* tidak hanya memecah teks secara semantik, tetapi juga mempertahankan hierarki dokumen melalui pemberian URI pada setiap *chunk* sehingga dapat ditelusuri kembali ke lokasi asalnya. Pada tahap ini dibentuk dua lapisan graf berikut,

- (a) *Document KG* (DKG) yang memetakan hubungan hierarkis dokumen (bab–subbab–*chunk*).
- (b) *Information KG* (IKG) yang menghubungkan antar-*chunk* melalui *keywords* untuk meningkatkan relevansi semantik pada dokumen.

Seluruh URI *chunk* juga disimpan pada basis data vektor untuk memungkinkan pencarian hibrid antara *dense embedding* dan hubungan semantik.

2. Pencarian Hibrid Berbasis Graf (*Retrieval*)

Setelah dilakukan *initial retrieval* berbasis *dense embedding*, sistem memperluas pencarian melalui operasi graf yang memanfaatkan struktur dokumen dan hubungan semantik. Tiga strategi yang digunakan adalah:

- (a) *Informed Chapter Search* (ICS), yaitu menambahkan *chunks* lain yang berada dalam bab yang sama dengan hasil pencarian awal.
- (b) *Informed Keyword Search* (IKS), yaitu menambahkan *chunks* yang terhubung melalui *keywords* pada IKG.
- (c) *Uninformed Keyword Search* (UKS), yaitu mengekstraksi *keyword* langsung dari pertanyaan pengguna dan mencari seluruh *chunk* terkait pada KG.

3. Pembentukan Jawaban Berbasis Bukti (*Generation*)

Pada fase akhir, sistem menerapkan *pass condition* untuk membatasi jumlah konteks yang diberikan pada LLM sehingga mencegah kelebihan informasi yang berpotensi menurunkan kualitas jawaban. Setelah konteks yang relevan

dipilih, proses *prompt engineering* dilakukan untuk memastikan model menghasilkan jawaban yang benar-benar berbasis bukti. Dalam hal ini, LLM diarahkan untuk menyusun jawaban dengan mengutip secara langsung bagian teks dari *chunk* yang relevan, menyatakan secara eksplisit apabila informasi yang tersedia tidak mencukupi untuk menjawab pertanyaan, serta tidak menyertakan pengetahuan eksternal yang tidak berasal dari konteks masukan pengguna. Pendekatan tersebut meningkatkan tingkat *faithfulness*, meminimalkan risiko *hallucination*, sekaligus memastikan transparansi jejak sumber yang mendukung reliabilitas hasil.

II.5.3 GraphRAG: Integrasi *Knowledge Graph* dan *Large Language Models*

Large Language Models (LLM) dan *Knowledge Graphs* (KG) merupakan dua paradigma representasi pengetahuan yang saling melengkapi. LLM unggul dalam pemrosesan bahasa alami serta generalisasi lintas tugas tetapi rentan terhadap halusinasi fakta. Sebaliknya, KG menawarkan representasi pengetahuan faktual dalam bentuk struktur graf eksplisit yang kuat untuk penalaran simbolik namun kurang mampu menangani pemahaman bahasa alami yang kompleks (Pan dkk. 2024).

Untuk mengatasi kelemahan LLM tersebut, pendekatan *Retrieval-Augmented Generation* (RAG) konvensional sering digunakan. Namun, RAG tradisional bekerja dengan cara memecah dokumen menjadi potongan teks (*chunks*) independen sehingga sering kali menghilangkan informasi struktural dan relasi esensial antar-entitas (Ma dkk. 2025). Sebagai solusi komprehensif atas keterbatasan tersebut, muncullah konsep *Graph Retrieval-Augmented Generation* (GraphRAG).

Berbeda dengan pencarian semantik biasa berbasis vektor teks, GraphRAG memanfaatkan mekanisme penelusuran graf (*graph traversal*) untuk mengambil konteks terstruktur secara utuh sebelum diberikan ke LLM (Han dkk. 2024). Mekanisme ini secara langsung mengekstraksi pengetahuan relevan dari data graf dengan merangkai subgraf atau jalur relasional berdasarkan kueri pengguna. Proses ini mencakup ekstraksi subgraf, penyaringan jalur (*path-filtering*), dan penyempurnaan jalur (*path-refinement*) guna menyajikan konteks penalaran yang akurat (Ma dkk. 2025).

Secara formal, integrasi generatif ini dimodelkan sebagai fungsi yang bersyarat terhadap pengetahuan graf:

$$p_{\theta}(y \mid x, \mathcal{K}) = \sum_{z \in \mathcal{Z}(x, \mathcal{K})} p_{\theta}(y \mid x, z) p(z \mid x, \mathcal{K}), \quad (\text{II.1})$$

dengan $\mathcal{K} = \{(h, r, t) \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}\}$ merepresentasikan KG sebagai himpunan

triples, dan $\mathcal{Z}(x, \mathcal{K})$ merupakan himpunan subgraf atau fakta terstruktur yang ditarik dari graf berdasarkan kueri x . Dalam skema ini, KG bertindak sebagai ruang pengetahuan non-parametrik penyedia panduan penalaran (*reasoning guidelines*), sedangkan LLM bertindak sebagai generator parametrik (Ma dkk. 2025).

Pendekatan GraphRAG ini menjadi sangat krusial dalam domain berskala kompleks seperti manajemen konfigurasi Kubernetes. Dengan memetakan hierarki objek dan referensi silang ke dalam graf, algoritma *traversal* dapat merangkai jejak penalaran (*reasoning path*) yang kohesif. Injeksi konteks terstruktur ini ke dalam perintah LLM terbukti secara efektif menekan tingkat halusinasi, memastikan akurasi faktual, dan mempertahankan validitas sintaksis pada hasil akhir manifes YAML yang dibangkitkan.

II.6 Evaluasi Sistem Model

Kinerja sistem GraphRAG dievaluasi menggunakan kerangka pengujian tiga dimensi yang secara holistik mengukur kualitas luaran, presisi pencarian, dan kemampuan penalaran model. Total pembobotan evaluasi didistribusikan ke dalam *Answer Quality* (40%), *Retrieval Quality* (35%), dan *Reasoning Quality* (25%).

II.6.1 Answer Quality (AnsQ)

Dimensi *Answer Quality* memiliki bobot evaluasi tertinggi sebesar 40% guna memastikan keluaran sistem tidak hanya relevan secara semantik tetapi juga dapat dieksekusi secara langsung pada lingkungan Kubernetes. Evaluasi pada dimensi ini mencakup lima metrik utama:

1. Syntactic Validity

Mengukur persentase keluaran yang memiliki struktur sintaksis YAML sempurna tanpa memunculkan galat saat diurai menggunakan pustaka `pyyaml`.

- **Formula:** $\frac{\text{valid_yaml}}{\text{total_yaml}} \times 100\%$
- **Kriteria Baik:** Dikatakan **baik** apabila menyentuh target mutlak 100% (tanpa cacat lekukan atau format).
- **Kriteria Buruk:** Dikatakan **buruk** apabila bernilai di bawah target tersebut karena dokumen pasti gagal dibaca oleh sistem infrastruktur.

2. Schema Compliance

Menghitung tingkat kelengkapan *required fields* dan ketepatan tipe data berdasarkan spesifikasi OpenAPI v1.30 menggunakan skema Pydantic. Pemenuhan metrik ini secara otomatis mencakup validasi *accuracy matching*.

- **Formula:** $\frac{\text{required_fields_present}}{\text{required_fields_total}} \times 100\%$
- **Kriteria Baik:** Dikatakan **baik** apabila mencapai target $\geq 95\%$.

- **Kriteria Buruk:** Dikatakan **buruk** apabila banyak atribut esensial terlewatkan sehingga memicu penolakan skema tingkat klaster.

3. *Kubernetes Validity*

Menguji kelayakan eksekusi manifes secara langsung ke *API server* klaster melalui perintah *dry-run*. Ini merupakan pengujian *domain-specific requirement* untuk memastikan validitas operasional dokumen.

- **Formula:** $\frac{\text{dry_run_success}}{\text{total_yaml}} \times 100\%$
- **Kriteria Baik:** Dikatakan **baik** apabila mencapai target $\geq 90\%$.
- **Kriteria Buruk:** Dikatakan **buruk** apabila struktur lulus uji teks tetapi ditolak oleh logika internal API Kubernetes.

4. *Faithfulness Score*

Menilai seberapa sesuai nilai-nilai di dalam YAML terhadap fakta yang ditarik dari *knowledge graph* menggunakan penilai *Large Language Model* (LLM-as-a-judge) (Es dkk. 2023).

- **Formula:** $\frac{\text{kg_sourced_values}}{\text{total_values}} \times 100\%$
- **Kriteria Baik:** Dikatakan **baik** apabila mencapai target $\geq 85\%$.
- **Kriteria Buruk:** Dikatakan **buruk** apabila LLM menyisipkan terlalu banyak informasi fiktif dari luar konteks spesifikasi.

5. *Answer Relevance*

Mengukur tingkat relevansi antara struktur YAML yang dibangkitkan dan maksud asli dari kueri pengguna (Es dkk. 2023).

- **Formula:** $\frac{\text{relevant_answers}}{\text{total_answers}} \times 100\%$
- **Kriteria Baik:** Dikatakan **baik** apabila mencapai target $\geq 85\%$, menandakan pemahaman sistem yang akurat.
- **Kriteria Buruk:** Dikatakan **buruk** apabila sistem menghasilkan manifes rapi tetapi tidak sesuai dengan sumber daya yang diminta pengguna.

II.6.2 *Retrieval Quality (RetQ)*

Dimensi *Retrieval Quality* dengan bobot 35% berfokus murni pada evaluasi kemampuan algoritma penelusuran graf dalam menarik konteks skema spesifikasi dokumentasi.

1. **F1-Score@k**

Merupakan nilai antara *precision* dan *recall* dari *node* yang berhasil ditarik. Metrik ini lebih bernilai operasional dibandingkan mengukur keduanya secara terpisah (Zhao dkk. 2024).

- **Formula:** $2 \times \frac{\text{Precision@k} \times \text{Recall@k}}{\text{Precision@k} + \text{Recall@k}}$
- **Kriteria Baik:** Dikatakan **baik** apabila mencapai target ≥ 0.75 .
- **Kriteria Buruk:** Dikatakan **buruk** apabila bernilai rendah, menandakan

an kueri menarik terlalu banyak konteks tidak relevan (*noise*) atau membuang terlalu banyak konteks penting.

2. **NDCG@k (Normalized Discounted Cumulative Gain)**

Mengevaluasi seberapa optimal sistem menempatkan skema paling relevan di urutan peringkat teratas sebelum dikirimkan ke *prompt* LLM (Es dkk. 2023).

- **Formula:** $\frac{DCG@k}{IDCG@k}$ (dengan pembobotan logaritmik)
- **Kriteria Baik:** Dikatakan **baik** apabila mencapai target ≥ 0.70 .
- **Kriteria Buruk:** Dikatakan **buruk** apabila informasi penting justru berada di urutan bawah sehingga berpotensi terpotong oleh batasan token LLM.

3. **Graph Coverage**

Menghitung persentase cakupan *node* relevan yang berhasil dijangkau oleh algoritma kueri graf.

- **Formula:** $\frac{\text{retrieved_nodes}}{\text{total_relevant_nodes}} \times 100\%$
- **Kriteria Baik:** Dikatakan **baik** apabila mencapai target $\geq 80\%$.
- **Kriteria Buruk:** Dikatakan **buruk** apabila algoritma gagal memetakan objek utama penyusun infrastruktur.

4. **Edge Coverage**

Merupakan metrik pengujian spesifik GraphRAG penilai interaksi topologi graf guna memastikan aturan semantik antar-objek terimplementasi dengan baik selama proses penarikan informasi.

- **Formula:** $\frac{\text{semantic_edges_triggered}}{\text{total_semantic_rules}} \times 100\%$
- **Kriteria Baik:** Dikatakan **baik** apabila mencapai target $\geq 75\%$, membuktikan pemahaman struktural berfungsi.
- **Kriteria Buruk:** Dikatakan **buruk** apabila sistem kesulitan melacak dependensi hierarkis antar-*node*.

II.6.3 Reasoning Quality (ReaQ)

Dimensi *Reasoning Quality* dengan bobot 25% mengevaluasi kapabilitas sistem dalam merangkai alur penalaran dan menghubungkan relasi referensial berlapis di dalam spesifikasi Kubernetes.

1. **Hop-Accuracy**

Menilai ketepatan algoritma dalam melintasi jalur relasional yang sesuai untuk *multi-hop reasoning*.

- **Formula:** $\frac{\text{correct_paths}}{\text{total_multi_hop_queries}} \times 100\%$
- **Kriteria Baik:** Dikatakan **baik** apabila mencapai target $\geq 80\%$.
- **Kriteria Buruk:** Dikatakan **buruk** apabila sistem sering tersesat mene-lusuri cabang graf yang tidak berhubungan dengan kueri.

2. *Multi-Hop Success Rate*

Mengukur persentase keberhasilan sistem dalam memberikan jawaban YAML utuh pada kueri kompleks penuntut penelusuran lebih dari satu lompatan graf.

- **Formula:** $\frac{\text{multi_hop_answered}}{\text{total_multi_hop_queries}} \times 100\%$
- **Kriteria Baik:** Dikatakan **baik** apabila mencapai target $\geq 75\%$.
- **Kriteria Buruk:** Dikatakan **buruk** apabila sistem hanya mampu merespons instruksi tunggal berskala sederhana.

3. *Hallucination Rate*

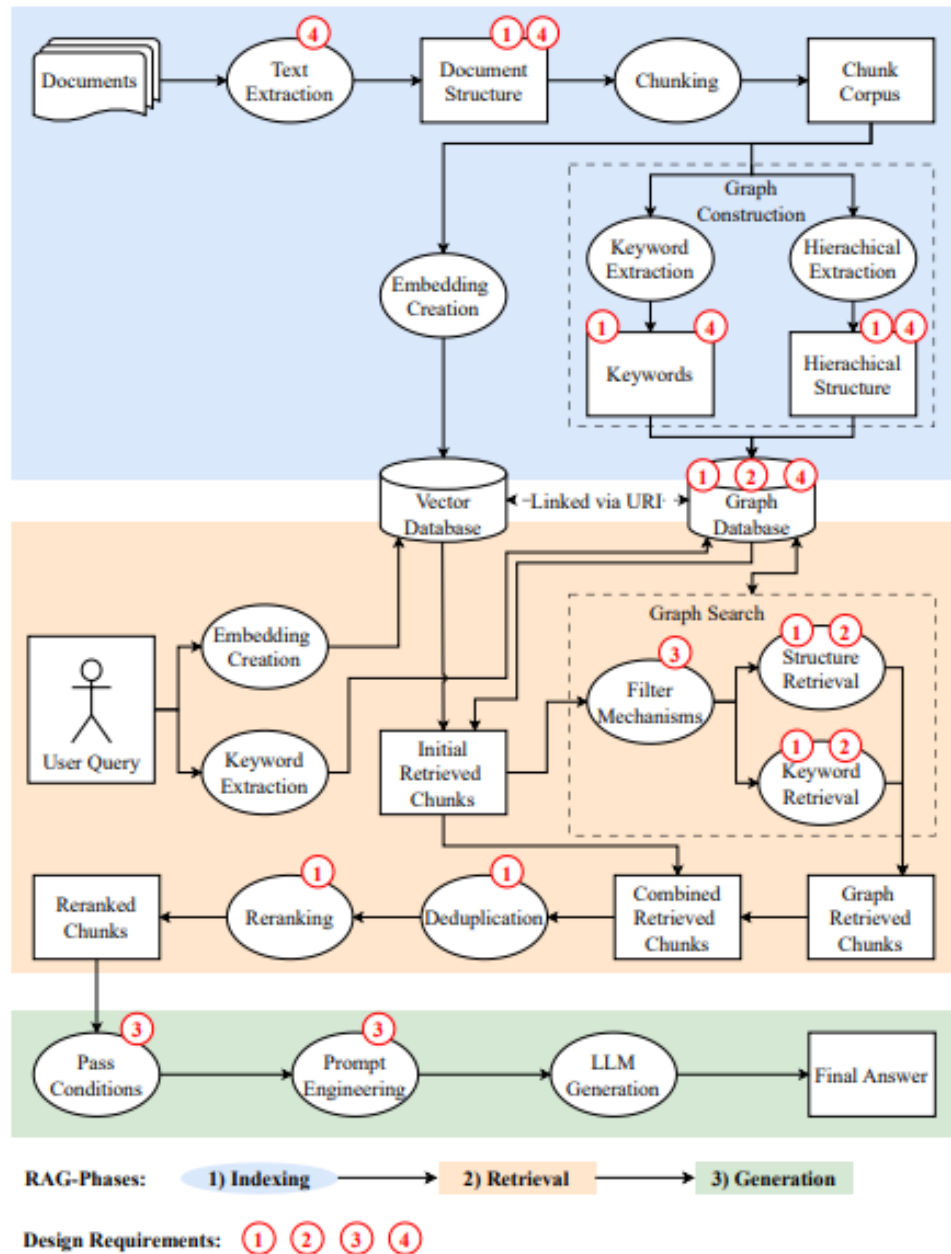
Memantau persentase kemunculan atribut fabrikasi atau struktur fiktif yang tidak bersumber dari dokumentasi resmi OpenAPI (Ji dkk. 2023).

- **Formula:** $\frac{\text{hallucinated_facts}}{\text{total_facts}} \times 100\%$
- **Kriteria Baik:** Makin rendah skor ini makin baik. Dikatakan **sangat baik** apabila $\leq 10\%$.
- **Kriteria Buruk:** Dikatakan **buruk** apabila melonjak melebihi target tersebut karena memicu kerentanan operasional.

4. *Scope Accuracy*

Menguji ketepatan klasifikasi model terhadap batasan sumber daya spesifik Kubernetes (misalnya perbedaan ketat antara sumber daya *Cluster-scoped* dan *Namespaced*).

- **Formula:** $\frac{\text{correct_scope_classification}}{\text{total_resources}} \times 100\%$
- **Kriteria Baik:** Dikatakan **baik** apabila mencapai target $\geq 95\%$.
- **Kriteria Buruk:** Dikatakan **buruk** apabila model menempatkan konfigurasi lintas ruang nama secara serampangan.



Gambar II.3 Model arsitektur integrasi KG dalam RAG (Knollmeyer, Caymazer, dan Grossmann 2025)

BAB III

ANALISIS MASALAH

III.1 Analisis Kondisi Saat Ini

Bagian ini menganalisis permasalahan yang muncul dalam proses pengembangan sistem *cloud-native* berbasis Kubernetes, ditinjau dari dua sudut utama yaitu pemahaman masalah bisnis (*Business Understanding*) dan kondisi data yang menjadi sumber permasalahan (*Data Understanding*).

III.1.1 Pemahaman Masalah Bisnis (*Business Understanding*)

Transformasi digital di berbagai industri mendorong adopsi arsitektur *cloud-native* berbasis *microservices* yang dikelola melalui Kubernetes. Dalam praktiknya, Kubernetes tidak hanya berfungsi sebagai alat operasional, tetapi juga sebagai aset strategis yang menentukan kecepatan dan stabilitas siklus pengembangan perangkat lunak (*software development lifecycle*). Kompleksitas konfigurasi yang tinggi, kurva pembelajaran yang curam, serta kebutuhan untuk melakukan *onboarding* bagi *engineer* baru menjadikan Kubernetes sebuah titik krusial dalam kapasitas operasional.

Permasalahan terkait pengelolaan konfigurasi Kubernetes tidak hanya bersifat teknis, tetapi menimbulkan dampak bisnis berupa biaya operasional dan risiko produksi. Beberapa permasalahan utama yang diidentifikasi adalah sebagai berikut:

- 1. B01 - Inefisiensi Waktu (*Operational Cost*)**

Dokumentasi Kubernetes bersifat luas dan terfragmentasi sehingga *developer/DevOps* membutuhkan waktu yang cukup lama untuk menelusuri relasi antar objek konfigurasi. Waktu pencarian informasi yang berlebih menjadi pemborosan sumber daya, terutama pada posisi *engineer* dengan biaya tenaga kerja tinggi.

- 2. B02 - Kesenjangan Pengetahuan (*Knowledge Gap*)**

Dokumentasi statis tidak sepenuhnya mendukung cara berpikir asosiatif manusia ketika memahami struktur spesifikasi. Sistem generatif berbasis LLM pun masih sering menghasilkan *hallucinated fields* karena tidak memiliki pemahaman skematik. Hal ini bisa memperburuk risiko kesalahan konfigurasi.

3. B03 - Risiko Kesalahan Konfigurasi (*Operational Risk*)

Kesalahan sintaksis atau penempatan atribut YAML yang tidak sesuai spesifikasi dapat menyebabkan *deployment failure* hingga *downtime*. *Downtime* produksi berimplikasi langsung pada kerugian finansial, reputasi layanan, dan penundaan proses bisnis.

Ketiga butir *business understanding* tersebut menjadi dasar dalam penyusunan rumusan masalah penelitian. B01 mengenai inefisiensi waktu akibat dokumentasi yang terfragmentasi mengarah pada kebutuhan representasi pengetahuan yang lebih terstruktur, sehingga dirumuskan sebagai Rumusan Masalah 1 tentang rekonstruksi spesifikasi OpenAPI menjadi *knowledge graph*. B02 yang menyoroti kesenjangan pemahaman skematik dan risiko *hallucination* pada LLM memunculkan Rumusan Masalah 2 terkait perancangan mekanisme *graph-guided retrieval* untuk menghadirkan konteks struktural yang akurat. Sementara itu, B03 terkait risiko kesalahan konfigurasi dan dampak operasionalnya diterjemahkan menjadi Rumusan Masalah 3 yang berfokus pada evaluasi efektivitas GraphRAG dalam meningkatkan relevansi konteks, *faithfulness*, serta validitas sintaksis dibandingkan pendekatan *baseline*.

III.1.2 Pemahaman Data (*Data Understanding*)

Bagian ini menjelaskan karakteristik data dokumentasi API Kubernetes yang menjadi sumber utama dalam proses konstruksi *Graph-based Retrieval Augmentation*. Analisis mencakup identitas data, struktur, pola hubungan antar objek, keterbatasan kualitas, serta seleksi data relevan yang digunakan dalam penelitian.

1. Sumber dan Spesifikasi Data (*Data Source & Specifications*)

Data penelitian diperoleh dari repositori resmi Kubernetes pada GitHub dengan alamat `kubernetes/kubernetes`. Data diambil dalam bentuk berkas `swagger.json` yang mengikuti standar *OpenAPI Specification* (OAS). Penelitian ini menggunakan versi Kubernetes **v1.30** sebagai acuan. Berkas `swagger.json` memiliki ukuran rata-rata 3,757 MB dan memuat 2.500–3.500 entitas definisi API. Kompleksitas tersebut menjustifikasi penggunaan metode ekstraksi otomatis, karena pendekatan manual berpotensi menghasilkan kesalahan interpretasi struktur dan kehilangan relasi semantik antar objek.

2. Analisis Struktur Dokumen

Berdasarkan inspeksi terhadap struktur `swagger.json`, terdapat dua komponen utama:

- (a) **Definitions (atau *components/schemas*)** : Mendeskripsikan spesifikasi objek seperti *Pod*, *Service*, dan *Deployment*.
- (b) **Paths** : Mendeskripsikan *endpoint* API, misalnya `GET /api/v1/pods`, sebagai interface interaksi objek dalam ekosistem Kubernetes.

Struktur tersebut diperkuat dengan mekanisme referensi `$ref` yang berfungsi untuk menghubungkan objek tanpa redundansi penulisan. Sebagai contoh, atribut `spec` dapat dideklarasikan dengan cara berikut pada objek `Deployment`:

```
"$ref": "#/definitions/io.k8s.api.apps.v1.DeploymentSpec"
```

Mekanisme ini menunjukkan bahwa data dokumentasi Kubernetes secara intrinsik bersifat graf karena terdapat simpul (objek) dan relasi (referensi silang).

3. Analisis Keterhubungan Data

Bagian ini menguraikan pola relasi antarkomponen Kubernetes sebagaimana tercermin dalam struktur `swagger.json`. Pola-pola ini menjadi dasar pembentukan relasi *node-edge* pada *Knowledge Graph* yang dibangun. Secara umum terdapat empat jenis hubungan yang teridentifikasi, yaitu (1) atomik pada Pod, (2) hierarkis, (3) *loose coupling*, dan (4) injeksi konfigurasi.

(a) Sifat Atomik pada Pod

Berbeda dengan komponen lain, Pod tidak memiliki relasi eksplisit yang menunjuk objek lain melalui skema maupun referensi nama. Pod bertindak sebagai unit eksekusi atomik yang menjadi tujuan dari beberapa mekanisme relasi, seperti seleksi oleh Service (*label matching*), injeksi konfigurasi dari `ConfigMap` dan `Secret`, serta pembungkus hierarkis oleh `Deployment`. Oleh karena itu, relasi Pod bersifat pasif, yakni tidak timbul dari atribut Pod itu sendiri, tetapi dari objek lain yang mengacu kepadanya.

(b) Hubungan Hierarkis

Objek `Deployment` tidak mereferensikan Pod melalui ID, tetapi melalui struktur bersarang. Relasi ini muncul pada:

```
spec.template
```

Bagian `template` memuat `PodSpec` secara lengkap sehingga hubungan `Deployment` → Pod dikategorikan sebagai relasi hierarkis (*nested composition*). Ekstraksi graf harus melakukan *deep traversal* pada elemen ini untuk menghasilkan node Pod beserta seluruh propertinya.

(c) Hubungan Loose Coupling

Relasi jaringan tidak bersifat hierarkis, tetapi bergantung pada pencocokan nilai atau referensi nama.

- **Service ke Pod** menggunakan pencocokan label:

```
spec.selector: { app: nginx }
```

Sistem harus mencari Pod lain dengan `metadata.labels` serupa.

- **Ingress ke Service** menggunakan referensi nama eksplisit:

```
backend.service.name: "frontend-service"
```

Sistem harus mencocokkan nilai tersebut dengan `metadata.name` milik Service.

(d) **Hubungan Injection**

Pod tidak menyimpan nilai konfigurasi secara langsung, tetapi mengambilnya dari node eksternal melalui referensi nama. Relasi ini bersifat satu arah (Pod → ConfigMap/Secret) dan ditandai oleh pola penggunaan pada Kode III.1:

Listing III.1 Contoh konfigurasi YAML

```
1  env:
2    valueFrom:
3      configMapKeyRef:
4        name: "app-config"
5
```

serta bentuk lainnya seperti Kode III.2:

Listing III.2 Contoh konfigurasi YAML dengan Secret

```
1  env:
2    valueFrom:
3      secretKeyRef:
4        name: "db-secret"
5
```

Selain itu, referensi juga dapat muncul melalui `envFrom.configMapRef`, `volume.configMap`, dan `volume.secret`. Mekanisme ini menciptakan relasi injeksi konfigurasi, yaitu nilai dari ConfigMap/Secret diambil dan dimasukkan ke lingkungan eksekusi Pod.

4. **Karakteristik Komponen Target**

Berdasarkan batasan masalah, penelitian ini hanya memfokuskan ekstraksi data pada enam entitas utama yang digunakan untuk membentuk arsitektur aplikasi bersifat *stateless*. Analisis struktur JSON dari masing-masing objek dilakukan untuk mengidentifikasi pola relasi serta properti penting yang relevan bagi konstruksi graf.

Tabel III.1 Karakteristik Komponen Target Kubernetes

Komponen	Lokasi Definisi (OpenAPI Key)	Deskripsi Spesifikasi
Deployment	<code>io.k8s.api.apps.v1.Deployment</code>	Properti Kunci: <code>spec.replicas</code>, <code>spec.selector</code>, <code>spec.template</code>. Pola Relasi: Hierarkis. Deployment membungkus Pod melalui field <code>template</code>.
Pod	<code>io.k8s.api.core.v1.Pod</code>	Properti Kunci: <code>metadata.labels</code>, <code>spec.containers</code>, <code>spec.volumes</code>. Pola Relasi: Atomik. Unit terkecil berisi definisi container, port, dan konfigurasi eksekusi.
Service	<code>io.k8s.api.core.v1.Service</code>	Properti Kunci: <code>spec.ports</code>, <code>spec.selector</code>, <code>spec.type</code>. Pola Relasi: <i>loose coupling</i> dengan <i>label matching</i>. Menentukan Pod target melalui kecocokan selector dengan labels.
Ingress	<code>io.k8s.api.networking.v1.Ingress</code>	Properti Kunci: <code>spec.rules</code>, <code>spec.backend.service.name</code>. Pola Relasi: <i>loose coupling</i>. Menunjuk Service melalui string nama secara eksplisit.

bersambung ke halaman berikutnya

Tabel III.1 Karakteristik Komponen Target Kubernetes (lanjutan)

Komponen	Lokasi Definisi (OpenAPI Key)	Deskripsi Spesifikasi
ConfigMap	<code>io.k8s.api.core.v1.ConfigMap</code>	Properti Kunci: <code>data</code>, <code>metadata.name</code>. Pola Relasi: <i>Injection</i>. Dipanggil oleh Pod via <code>configMapRef</code> atau <code>envFrom</code>.
Secret	<code>io.k8s.api.core.v1.Secret</code>	Properti Kunci: <code>data</code> (<code>base64</code>), <code>type</code>. Pola Relasi: <i>Injection</i>. Digunakan Pod via <code>secretKeyRef</code>.

5. Penilaian Kualitas dan Keterbatasan Data

Analisis menunjukkan beberapa isu kualitas data yang relevan terhadap sistem RAG, antara lain:

- Isu *deep nesting***, yaitu beberapa atribut berada hingga tingkat kedalaman 5–6 lapisan, misalnya:
`spec.template.spec.containers.resources.limits`
Kedalaman ini berpotensi mengurangi kemampuan *vector embedding* dalam menjaga konteks semantik.
- Isu ambiguitas leksikal**, yaitu kata kunci seperti `spec` muncul ribuan kali dan dapat merujuk pada puluhan entitas berbeda. Hal ini membuktikan bahwa pendekatan pencarian berbasis kata kunci tidak memadai tanpa interpretasi struktur skematik.

III.2 Analisis Kebutuhan

Tahap analisis kebutuhan berfokus pada identifikasi kapabilitas yang harus dimiliki sistem guna menjawab tujuan penelitian dan kebutuhan bisnis. Untuk memastikan bahwa kebutuhan yang dirumuskan bersifat relevan dengan masalah yang dihadapi, dilakukan analisis *technical GAP* pada pendekatan eksisting yang ada pada sistem *Retrieval-Augmented Generation* pada Tabel III.2.

Tabel III.2 *Technical Gap Analysis* pada Domain Kubernetes

Aspek Evaluasi	Kondisi Saat Ini (<i>Existing Approaches</i>)	Kesenjangan di Domain Kubernetes
T01 - Representasi Struktur Data	Menggunakan pemecahan teks secara <i>linear chunking</i> , yaitu dokumen hierarkis dipecah menjadi potongan terpisah sehingga hubungan <i>parent-child</i> terputus (cao2025neusymrag).	Terjadi fenomena <i>loss of deep context</i> . Sistem gagal memahami kedalaman hierarki, misalnya properti <i>limits</i> merupakan turunan bersarang dari Deployment, karena strukturnya terpisah antar <i>chunk</i> (Kotstein dan Decker 2024).
T02 - Mekanisme Validasi	Validasi bergantung pada model LLM atau aturan manual berbasis <i>hard constraints</i> yang harus ditulis satu per satu (Wan dkk. 2025).	Risiko terjadinya <i>syntactic hallucination</i> tinggi (LLM mengarang <i>field</i>), atau beban pemeliharaan menjadi besar saat aturan manual harus diperbarui (Gao dkk. 2024).
T03 - Ambiguitas Entitas	Mengandalkan frekuensi kemunculan kata kunci (<i>keyword frequency</i>) atau kedekatan vektor semata.	Terdapat ambiguitas pada pola <i>loose coupling</i> karena <i>keywords</i> (misal: <i>labels</i> , <i>ports</i>) muncul berulang di banyak objek berbeda. Hal ini menyebabkan tercampurnya konteks antar-objek (<i>contextual mixing</i>).
T04 - Pemeliharaan Pengetahuan	Membutuhkan <i>fine-tuning</i> ulang model atau pembaruan aturan manual ketika terjadi perubahan versi API (Gao dkk. 2024).	Tidak efisien, mudah menjadi <i>obsolete</i> , dan sensitif terhadap perubahan versi Kubernetes yang dirilis secara berkala (± 4 bulan).

bersambung ke halaman berikutnya

Tabel III.2 *Technical Gap Analysis* pada Domain Kubernetes (lanjutan)

Aspek Evaluasi	Kondisi Saat Ini (<i>Existing Approaches</i>)	Kesenjangan di Domain Kubernetes
T05 - Akurasi Jawaban	Sistem cenderung menghasilkan jawaban naratif umum dan tidak memberikan jaminan presisi sintaksis konfigurasi YAML.	Tidak mampu menghasilkan artefak konfigurasi yang sesuai karena sering ditemukan kesalahan format atau struktur.

III.2.1 Kebutuhan Fungsional

Berdasarkan *Business Understanding* (B01–B03), diidentifikasi sejumlah urgensi bisnis yang dianalisis lebih lanjut hingga menghasilkan beberapa celah teknis (*technical gaps*) yang menjadi dasar perumusan kebutuhan sistem. Setiap *technical gap* kemudian dijawab melalui perancangan kebutuhan fungsional yang akan diimplementasikan untuk menyelesaikan permasalahan bisnis yang ada.

Tabel III.3 berikut memetakan hubungan antara kebutuhan bisnis, *technical gap*, serta kebutuhan fungsional yang dirumuskan untuk mengatasinya.

Tabel III.3 Pemetaan Kebutuhan Bisnis terhadap *Technical Gap* dan Kebutuhan Fungsional

Business Requirement (B)	Technical GAP (T)	Functional Requirement (F)
4*B01 – Inefisiensi waktu	2*T01 – Representasi struktur data	F-01 <i>Structured Schema Representation</i>
		F-02 <i>Structured Relation Retrieval</i>
	2*T03 – Ambiguitas entitas	F-03 <i>Intent Classification</i>
		F-04 <i>Context Construction & Injection</i>
3*B02 – Kesenjangan pengetahuan	2*T02 – Mekanisme validasi	F-05 <i>Attribute Enrichment</i>
		F-06 <i>CLI Interaction Interface</i>
	T04 – Pemeliharaan pengetahuan	F-07 <i>Selective Components Parsing</i>
1*B03 – Risiko kesalahan konfigurasi	T05 – Akurasi jawaban	F-08 <i>Conditional YAML Generation Constraint</i>

Berdasarkan hasil pemetaan pada Tabel III.3, berikut adalah kebutuhan fungsional yang dibutuhkan oleh sistem,

Tabel III.4 Tabel harga bahan tertier

Nama	Satuan	Harga
Buku	Exemplar	25000
Komputer	Unit	2500000
Pensil	Buah	118900
Pensil	Buah	118900
Pensil	Buah	118900
Pensil	Buah	118900
Pensil	Buah	118900

III.2.2 Kebutuhan Non Fungsional

Kebutuhan non-fungsional mendefinisikan karakteristik kualitas yang harus dimiliki oleh sistem untuk memastikan kinerja, keandalan, dan keamanan operasional. Kebutuhan ini penting untuk menjamin bahwa sistem tidak hanya berfungsi dengan benar, tetapi juga memenuhi standar kualitas yang diharapkan oleh pengguna. Berikut adalah kebutuhan non-fungsional utama yang diidentifikasi:

Tabel III.5 Kebutuhan Non-Fungsional Sistem

Kode	Parameter	Deskripsi Spesifikasi
NF-01 (Reliability)	<i>Fail-Safe Mechanism</i>	Sistem harus mencegah keluaran halusinasi melalui mekanisme <i>error handling</i> , yaitu (1) Jika entitas tidak ditemukan, sistem harus memberi respons “Entity not found”. (2) Jika YAML terdeteksi tidak valid oleh validator internal, sistem harus menampilkan peringatan “Warning: Potential Syntax Error” pada antarmuka CLI pengguna.
NF-02 (Performance)	<i>Retrieval–Generation Latency</i>	Waktu total yang dibutuhkan untuk memproses satu permintaan pengguna (<i>end-to-end latency</i>) tidak boleh melebihi 60 detik pada lingkungan pengujian standar.
NF-03 (Usability)	<i>CLI Usability Standard</i>	Antarmuka <i>Command-Line Interface (CLI)</i> harus menyediakan pengalaman penggunaan yang konsisten, mudah dipahami, dan informatif.
NF-04 (Portability)	<i>Resource Constraints</i>	Seluruh komponen sistem harus dapat dijalankan pada lingkungan lokal (<i>Local Environment</i>) dengan spesifikasi perangkat keras terbatas (minimal RAM 8GB), tanpa ketergantungan pada infrastruktur <i>cloud</i> terdistribusi.

III.3 Alternatif Solusi

Bagian ini membahas berbagai alternatif solusi yang dapat diimplementasikan untuk memenuhi kebutuhan sistem yang telah diidentifikasi sebelumnya.

III.3.1 Hybrid KG–Vector RAG

Pendekatan yang diusulkan oleh Wan dkk. (2025) adalah *hybrid KG–Vector RAG* untuk *domain-centric Q&A* di bidang *smart manufacturing* (DfAM). Inti gagasan dari metode ini adalah menggabungkan dua jalur retrieval, yaitu (1) **vector-based RAG** yang efisien namun cenderung kurang akurat secara kontekstual, dan (2) **graph-based RAG** berbasis metadata *knowledge graph* yang mampu melakukan penalaran relasional tetapi kurang baik dalam bagian skala dan fleksibilitas. Alih-alih memilih salah satu, solusi ini membangun *pipeline* yang mengoptimalkan keduanya.

Secara metodologis, pendekatan ini dimulai dari pengumpulan korpus domain-spesifik yang kemudian diolah menjadi dua representasi paralel, yaitu (a) *vector store* hasil *chunking* teks dan pembuatan *embedding* (*text-embedding-3-small*) untuk retrieval semantik, dan (b) metadata-based KG di Neo4j yang memetakan entitas seperti dokumen, topik, dan kata kunci ke dalam tripel terarah. Lapisan KG ini tidak merepresentasikan struktur sintaksis kode, tetapi struktur konseptual.

Tahap berikutnya adalah *semantic alignment* dengan menyuntikkan *domain constraints* dan *schema constraints* ke dalam proses retrieval. *Hard constraint* digunakan untuk menjaga kebenaran faktual, sedangkan *schema constraint* membatasi jenis relasi yang boleh muncul sesuai ontologi domain.

Pada tahap **hybrid KG–Vector RAG**, *query* pengguna terlebih dahulu diekstraksi entitasnya (NER) yang kemudian dilakukan proses berikut, (1) dilakukan *node & relation retrieval* pada KG untuk mendapatkan subgraf relevan, dan (2) jika diperlukan jawaban lebih rinci, sistem menambahkan *vector similarity search* untuk mengambil *text chunks* yang paling dekat secara semantik. Kedua konteks ini lalu digabung dan diproses LLM (GPT-4o / GPT-4o mini / DeepSeek V2) dengan *prompt engineering* sebagai lapisan tambahan (KG–Vector + PE).

Dari sisi evaluasi, Wan dkk. (2025) mengukur tiga metrik generik RAG, yaitu *Exact Match* (EM), *Context Precision* (CP), dan *latency*. Pada konfigurasi terbaik (KG–Vector + *prompt enhancement* dengan GPT-4o), mereka melaporkan EM sebesar 77,8% dan CP sebesar 76,5%, mengungguli *vector-only RAG* dan *KG-only RAG*. Selain itu, studi ini juga menggunakan *LLM-as-a-judge* untuk menilai jawaban dari sisi akurasi teknis dan kesesuaian domain, dengan skor rata-rata tertinggi 7,83 untuk

kombinasi GPT-4o + KG-Vector + PE. Trade-off yang muncul adalah peningkatan *latency*: *vector-only* merupakan yang tercepat, sedangkan *KG-Vector* + *PE* paling akurat namun paling lambat karena overhead traversal KG dan *prompt refinement*.

III.3.2 Pendekatan *GNN-augmented GraphRAG* (G-Retriever)

Pendekatan G-Retriever merupakan arsitektur *retrieval-augmented generation* (RAG) yang dirancang secara khusus untuk pemahaman graf tekstual berskala besar dengan memadukan kemampuan *Graph Neural Networks* (GNN) dan *Large Language Models* (LLM). Model ini dikembangkan untuk menangani pertanyaan kompleks terhadap graf yang memiliki atribut tekstual, sekaligus mengurangi halusinasi melalui mekanisme *subgraph retrieval* yang terstruktur (he2024gretriever).

G-Retriever diperkenalkan untuk menjawab tantangan utama pada LLM ketika berhadapan dengan data bergraf besar, yaitu (1) keterbatasan konteks LLM terhadap graf yang memiliki ribuan *node* dan *edge*, (2) tingginya risiko halusinasi ketika model hanya bergantung pada representasi terkompresi, serta (3) kebutuhan untuk mengekstrak bagian graf yang paling relevan terhadap kueri. Berbeda dengan metode graf-konvensional atau RAG berbasis teks datar, G-Retriever tidak mengandalkan teks atau *embedding* saja, tapi mengandalkan topologi graf dalam proses *retrieval* nya sehingga hasil generasi didasarkan pada sub-graf yang valid dan dapat diverifikasi.

Secara umum, G-Retriever terdiri atas empat tahap utama, yaitu *indexing*, *retrieval*, *subgraph construction*, dan *generation* (he2024gretriever).

1. *Indexing*

Pada tahap ini, setiap node dan edge dalam graf diubah menjadi representasi numerik menggunakan model bahasa seperti Sentence-BERT. Atribut tekstual (misalnya label node, deskripsi, atau tipe relasi) diekstraksi dan diencode untuk disimpan dalam struktur indeks *nearest neighbor*. Proses ini memungkinkan sistem menemukan node yang paling relevan berdasarkan kemiripan semantik.

2. *Retrieval*

Kueri pengguna diubah ke dalam *embedding* dan dibandingkan menggunakan *cosine similarity* untuk menemukan k node dan *edge* paling relevan. Proses ini tidak hanya memfokuskan pencarian pada elemen graf yang memiliki hubungan semantik tinggi, tetapi juga memastikan bahwa proses retrieval tidak bergantung sepenuhnya pada generasi LLM sehingga meminimalkan potensi halusinasi.

3. *Subgraph Construction*

Tahap kritis dari G-Retriever adalah membangun sub-graf yang koheren menggunakan algoritma *Prize-Collecting Steiner Tree* (PCST). Algoritma ini memilih *node* dan *edge* yang paling informatif sambil menjaga ukuran sub-graf tetap kecil agar muat dalam konteks LLM. PCST memberikan mekanisme matematis yang menyeimbangkan antara “hadiah” (relevansi semantik) dan “biaya” (ukuran graf) sehingga hanya struktur yang benar-benar relevan yang dikirimkan kepada LLM.

4. *Generation*

Sub-graf hasil PCST kemudian diubah ke dalam bentuk tekstual melalui proses *textualization* dan diberikan kepada LLM sebagai bagian dari *prompt*. Sebuah *graph encoder* (misalnya GAT atau Graph Transformer) digunakan untuk menghasilkan *graph token* yang berfungsi sebagai *soft prompt*. LLM tanpa *fine-tuning* kemudian menghasilkan jawaban yang selaras dengan struktur dan konteks sub-graf.

Hasil eksperimen yang disampaikan dalam dokumen menunjukkan bahwa G-Retriever meningkatkan performa signifikan di berbagai skenario graf. Temuan pentingnya meliputi:

1. Peningkatan akurasi. Dalam konfigurasi *frozen LLM*, G-Retriever mencapai peningkatan rata-rata 30–47% dibanding metode *prompt tuning* graf konvensional.
2. Efisiensi skala besar. Jumlah token dapat dikurangi hingga 83–99% setelah tahap *retrieval*, memungkinkan LLM bekerja pada graf besar tanpa kehilangan konteks penting.
3. Mitigasi halusinasi. Validitas node meningkat dari 31% menjadi 77% dan validitas edge dari 12% menjadi 76%. Hal ini menunjukkan bahwa pengambilan data langsung dari graf secara substansial mengurangi risiko halusinasi.

Secara keseluruhan, G-Retriever memberikan fondasi kuat sebagai alternatif solusi untuk penelitian ini. Pendekatan ini menonjol karena menggabungkan *retrieval* berbasis struktur graf dan generasi LLM secara efektif.

III.3.3 *Hybrid Neural-Symbolic dengan Relational DB (Pendekatan NeuSym-RAG)*

NeuSym-RAG menggabungkan model neural dengan basis pengetahuan simbolik berbasis aturan ([cao2025neusymrag](#)). Penelitian tersebut menggunakan basis data relasional (*relational DB*) untuk menyimpan entitas dan atribut dalam format tabel yang disusun secara terstruktur. Sistem ini kemudian mengeksekusi aturan simbolik

(*symbolic rules*) untuk memvalidasi hasil inferensi LLM secara deterministik.

Jika diadaptasikan untuk Kubernetes, skema tabel dapat dibuat untuk menyimpan objek seperti:

1. Tabel *Workloads* berisi Pod, Deployment, beserta *replica specifications*;
2. Tabel *Networking* berisi Service, Ingress beserta *selector-labelling rules*;
3. Tabel *Configuration* berisi ConfigMap dan Secret.

Validasi konfigurasi YAML kemudian dilakukan melalui aturan simbolik, misalnya:

“Jika `Service.selector` tidak cocok dengan `Pod.labels`, maka YAML tidak valid.”

Meskipun dapat menghasilkan validasi deterministik yang kuat, pendekatan *Relational DB* memiliki kelemahan utama untuk domain Kubernetes, yaitu banyaknya operasi *join* yang diperlukan untuk menelusuri relasi antar objek yang tersusun hierarkis dalam OpenAPI JSON. Kompleksitas ini mengakibatkan kinerja kueri tidak efisien ketika relasi melibatkan lebih dari dua tingkat kedalaman (*multi-hop*), seperti pada kasus hubungan `Deployment → PodTemplate → Container → ConfigMap/Secret`.

III.4 Analisis Pemilihan Solusi

Bagian ini bertujuan menentukan pendekatan pemecahan masalah yang paling sesuai untuk membangun sistem *Graph-based Retrieval* pada dokumentasi API Kubernetes. Evaluasi dilakukan dengan membandingkan beberapa alternatif solusi berdasarkan kriteria yang disesuaikan dengan karakteristik konfigurasi *cloud-native* yang bersifat deklaratif, terstruktur, dan saling bergantung secara hierarkis.

III.4.1 Rubrik Evaluasi Solusi

Evaluasi dilakukan menggunakan skala 1 (Kurang Baik) hingga 5 (Sangat Baik), berdasarkan empat kriteria utama sebagai berikut:

K1. Kesesuaian Representasi Data

Mengukur kemampuan metode dalam menangani data konfigurasi yang bersifat hierarkis dan memiliki *cross-reference* antar objek (misalnya *Service → Pod* melalui *selector*).

K2. Kemampuan Generatif

Menilai kemampuan sistem menghasilkan kode YAML yang baru dan valid, bukan sekadar menemukan lokasi informasi pada dokumentasi.

K3. Fleksibilitas Pemeliharaan

Mengukur kemudahan sistem dalam melakukan pembaruan pengetahuan ke-

tika spesifikasi API Kubernetes berubah versi tanpa memerlukan *fine-tuning* ulang.

K4. Presisi Konteks

Menilai kemampuan sistem menekan halusinasi melalui pembatasan struktural yang tegas, terutama pada relasi antar objek konfigurasi.

Tabel III.6 menunjukkan kategori penilaian untuk masing-masing skor pada rubrik evaluasi tersebut.

Tabel III.6 Rubrik Penilaian Kriteria Evaluasi Solusi

Skor	Kategori	Deskripsi
1	Kurang Baik	Tidak memenuhi kebutuhan domain Kubernetes secara fungsional maupun struktural.
2	Cukup Buruk	Hanya relevan sebagian dan berisiko tinggi menghasilkan kesalahan konfigurasi.
3	Cukup Baik	Dapat digunakan, tetapi memerlukan banyak penyesuaian untuk konteks Kubernetes.
4	Baik	Umumnya sesuai dan dapat diterapkan dengan penyesuaian minor.
5	Sangat Baik	Sangat sesuai dengan karakteristik masalah dan hampir tidak memerlukan modifikasi.

III.4.2 Matriks Keputusan & Justifikasi Penilaian

Penilaian dilakukan terhadap tiga pendekatan alternatif serta metode usulan. Hasil evaluasi ditampilkan pada Tabel III.7.

Tabel III.7 Matriks Perbandingan Solusi Berdasarkan Rubrik Evaluasi

Metode	K1	K2	K3	K4	Total
Hybrid KG–Vector RAG (Alt 1)	5	4	5	5	19
GNN-augmented GraphRAG (Alt 2)	5	3	4	5	17
NeuSym-RAG (Alt 3)	3	2	3	4	12

Berdasarkan hasil penilaian kuantitatif pada Tabel III.7, diperlukan penjelasan lebih lanjut mengenai dasar pemberian skor pada masing-masing kriteria. Berikut ini justifikasi kuantitatif terhadap nilai yang diberikan untuk setiap metode.

1. Hybrid KG–Vector RAG

- a. **K1 = 5:** Mampu menggabungkan struktur relasional (KG) dan teks tidak terstruktur (*vector store*). Cocok untuk OpenAPI Kubernetes yang bersifat hierarkis dan memiliki *cross-reference*.
- b. **K2 = 4:** Menghasilkan konteks yang lengkap sehingga LLM dapat menyusun YAML baru dengan lebih tepat, meskipun tidak memiliki mekanisme pembatasan struktural seketat GNN.
- c. **K3 = 5:** KG dapat diperbarui secara *incremental* sedangkan *vector store* hanya memerlukan *re-embedding* teks tanpa melatih ulang model.
- d. **K4 = 5:** Kombinasi *hard constraint*, *schema constraint*, dan *vector similarity* memberikan presisi konteks, serta terbukti meningkatkan EM dan CP pada studi.

2. GNN-augmented GraphRAG (G-Retriever)

- a. **K1 = 5:** Representasi graf Kubernetes sangat sesuai diproses oleh GNN yang sensitif terhadap struktur topologi dan relasi *multi-hop* (Deployment → PodTemplate → Container).
- b. **K2 = 3:** Kemampuan generatif bergantung pada sub-graf hasil PCST. LLM menghasilkan jawaban berdasarkan sub-graf, tetapi tidak menyusun struktur baru secara fleksibel.
- c. **K3 = 4:** Meski tidak memerlukan *fine-tuning* ulang, proses *indexing* dan pembentukan *embedding* graf harus diulang ketika versi API berubah.
- d. **K4 = 5:** Eksperimen menunjukkan validitas node meningkat dari 31% menjadi 77%, dan validitas edge dari 12% menjadi 76%, menunjukkan kontrol presisi yang sangat tinggi.

3. NeuSym-RAG (Hybrid Neural-Symbolic)

- a. **K1 = 3:** Representasi tabel relasional kurang mampu menampung kedalaman hierarki OpenAPI tanpa melakukan banyak operasi *join* sehingga tidak sebaik KG atau GNN.
- b. **K2 = 2:** Aturan simbolik bersifat validasi, bukan generasi. Kemampuan LLM untuk membuat YAML baru terbatas karena sistem lebih cocok untuk *consistency checking*.
- c. **K3 = 3:** Relational schema stabil, tetapi aturan simbolik harus diperbarui manual saat Kubernetes menambahkan spesifikasi baru.
- d. **K4 = 4:** Aturan deterministik (*symbolic rules*) memberi kontrol kuat terhadap halusinasi, tetapi kelenturannya tidak sekuat constraint pada KG–Vector atau pemodelan struktural pada GNN.

III.4.3 Analisis Pemilihan dan Penyesuaian (Selection & Adjustment)

Berdasarkan hasil evaluasi pada Tabel III.7, metode *Hybrid KG–Vector RAG* (Alt 1) memperoleh skor tertinggi di antara alternatif lain dan menjadi landasan utama dalam perancangan sistem. Pendekatan tersebut dipilih karena berhasil menggabungkan representasi relasional eksplisit dari KG dengan cakupan semantik luas dari *vector retrieval*. Hal ini relevan pada domain Kubernetes, yaitu satu objek seperti *Pod* tidak hanya memiliki struktur *rigid* tetapi juga semantik.

Namun, arsitektur Wan dkk. (2025) masih memiliki keterbatasan jika diterapkan langsung pada konteks API Kubernetes. Komponen *Semantic Alignment* pada penelitian tersebut sangat bergantung pada aturan manual (*domain constraints*) yang harus dituliskan eksplisit pada *prompt*. Pada Kubernetes, pendekatan ini tidak efisien karena terdapat ribuan *field* API yang berubah pada tiap versinya sehingga menulis aturan secara manual menjadi tidak praktis.

Oleh karena itu, penelitian ini mengusulkan penyesuaian strategi sebagai berikut:

- A1. Pendekatan Wan dkk. (2025) : Validasi berbasis aturan manual di dalam *prompt* (*Constraint-based Semantic Alignment*).
- A2. Implementasi solusi Kubernetes : Validasi berbasis struktur otomatis melalui penelusuran graf (*Graph-Native Structural Traversal*).

Pendekatan ini menjadikan topologi *Knowledge Graph* sebagai validator bawaan. Jika relasi antar objek ditemukan secara eksplisit pada graf (misalnya *Service* → *Deployment* melalui *selector*), maka konfigurasi tersebut dianggap valid tanpa perlu mendefinisikan aturan tambahan.

Kesimpulan: Solusi akhir yang dipilih adalah *GraphRAG with Structural Traversal*, yaitu adaptasi dari arsitektur *Hybrid KG–Vector RAG* yang menggantikan validasi aturan *hard constraints* dengan validasi relasional dari topografi graf. Pendekatan ini meningkatkan efisiensi pemeliharaan dan menekan risiko halusinasi konfigurasi Kubernetes.

BAB IV

PERANCANGAN

Ilustrasikan desain konsep solusi dalam bentuk model konseptual dan penjelasan secara ringkas, beserta perbedaannya dengan sistem saat ini. Ilustrasi harus dapat dibandingkan (*before and after*). Karena masih berupa proposal, bab ini hanya berisi gambar desain konsep solusi tersebut dan penjelasan perbandingannya dengan gambar sistem yang ada saat ini (yang tergambar di awal Bab III).

BAB V

IMPLEMENTASI

asdfasdf

BAB VI

EVALUASI

asdfasdf

VI.1 Metode Evaluasi

asdfasdf

VI.2 Hasil Evaluasi

asdfasdf

VI.3 Pembahasan Hasil Evaluasi

VI.4 Evaluasi Kesalahan Penulisan yang Sering Terjadi

VI.4.1 Penggunaan Kata "di mana" atau "dimana"

Banyak yang menuliskan kata "di mana" atau "dimana" sebagai pengganti kata "which" dalam bahasa Inggris. Padahal, penggunaan kata "di mana" atau "dimana" tidak tepat dalam konteks tersebut. Demikian juga untuk kata serupa, misalnya "yang mana". Kata "di mana" atau "dimana" ini harus diganti dengan kata lain, seperti "dengan", "tempat", "yang", dan sebagainya tergantung kalimatnya. Penjelasan lengkap dapat dilihat pada (BPBI).

VI.4.2 Penggunaan Kata "sedangkan" dan "sehingga"

Kata "sedangkan" dan "sehingga" adalah kata hubung atau konjungsi. Konjungsi adalah kata atau ungkapan yang menghubungkan satuan bahasa (kata, frasa, klausa, dan kalimat). Konjungsi dapat dibagi menjadi konjungsi intrakalimat dan antarkalimat. Kata "sedangkan" menghubungkan dua klausa yang bersifat kontrasif, sedangkan "sehingga" menghubungkan dua klausa yang bersifat kausal. Dalam ragam formal, kata hubung "sedangkan" dan "sehingga" hanya dapat digunakan sebagai konjungsi intrakalimat sehingga kedua konjungsi itu **tidak dapat diletakkan pada awal kalimat**. Selain itu, penggunaan kata "sedangkan" harus didahului oleh koma (,), sedangkan kata "sehingga" tidak perlu didahului oleh koma (,). Contoh penggunaan yang benar dan salah dapat dilihat pada Tabel VI.1.

Tabel VI.1 Contoh penggunaan kata "sedangkan" dan "sehingga"

Kata	Salah	Benar
sedangkan	Sedangkan sistem lama masih digunakan oleh banyak pengguna.	Sistem lama masih digunakan oleh banyak pengguna, sedangkan sistem baru belum siap.
sehingga	Sehingga sistem lama masih digunakan oleh banyak pengguna.	Sistem lama masih digunakan oleh banyak pengguna sehingga sistem baru belum siap.

VI.4.3 Penggunaan Istilah yang Tidak Baku

Ada beberapa istilah yang sering digunakan dalam pembicaraan sehari-hari, tetapi tidak baku dalam penulisan ilmiah. Beberapa istilah tersebut antara lain:

1. analisa → analisis
2. eksisting atau existing → yang ada atau saat ini
3. bisnis proses → proses bisnis
4. user → pengguna
5. system → sistem
6. database → basis data
7. aktifitas → aktivitas
8. efektifitas → efektivitas
9. sosial media → media sosial

VI.4.4 Pemisah Desimal dan Ribuan

Tanda pemisah desimal dalam bahasa Indonesia adalah tanda koma, contoh:

1. (Salah) Akurasi naik menjadi 50.6%
2. (Benar) Akurasi naik menjadi 50,6%

VI.4.5 Daftar atau *List*

Ada beberapa aturan penulisan daftar atau *list* yang perlu diperhatikan, antara lain:

- a) Jika memungkinkan, hindari penggunaan "bullet points" atau sejenisnya. Sebaiknya, gunakan angka (1, 2, 3, ...) atau huruf (a, b, c, ...). Dengan demikian, pembaca dapat dengan mudah melihat jumlah *item* atau *list*.
- b) Jika dalam daftar hanya ada satu item, tidak perlu menggunakan nomor urut.
- c) Penjelasan atau deskripsi suatu item sebaiknya menyatu dengan judul item tersebut, tidak berbeda halaman. Contoh yang salah: judul item ada di halaman 10, namun deskripsinya di halaman 11. Sebaiknya pindahkan judul tersebut ke halaman 11.

- d) Jika penjelasan atau deskripsi suatu item cukup panjang, misalnya lebih dari 1 halaman atau terdiri atas beberapa paragraf, sebaiknya setiap item tersebut dijadikan judul subbab, kecuali jika level subbab sudah mencapai level 4.

VI.4.6 Penggunaan Kata "masing-masing" dan "setiap"

Kata "masing-masing" digunakan di belakang kata yang diterangkan, misalnya "Setiap proses menggunakan algoritma masing-masing". Kata "tiap-tiap" atau "setiap" ditempatkan di depan kata yang diterangkan, misalnya "Setiap proses menggunakan algoritma tertentu".

BAB VII

PENUTUP

VII.1 Kesimpulan

Jelaskan secara detail langkah-langkah rencana selanjutnya, hal-hal yang diperlukan atau akan disiapkan, dan risiko dan mitigasinya, yang meliputi:

VII.2 Saran

asdfas

DAFTAR PUSTAKA

- Antonio Moreno-Cediel, David De-Fitero-Dominguez, Eva Garcia-Lopez. Antonio Garcia-Cabot. 2025. "Optimising retrieval performance in RAG systems: A new growing window semantic chunking strategy to address weak semantic boundaries". *Knowledge-Based Systems* 331:114896. <https://doi.org/10.1016/j.knosys.2025.114896>.
- Cloud Native Computing Foundation. 2023. *CNCF Annual Survey 2023*. Diakses pada November 22, 2025. <https://www.cncf.io/reports/cncf-annual-survey-2023/>.
- Es, Shahul, Jithin James, Luis Espinosa-Anke, dan Steven Schockaert. 2023. "RA-GAS: Automated Evaluation of Retrieval Augmented Generation". *arXiv preprint arXiv:2309.15217*.
- Gao, Yunfan, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, dan Haofen Wang. 2024. "Retrieval-Augmented Generation for Large Language Models: A Survey". Accessed November 25, 2025, *arXiv preprint arXiv:2312.10997*.
- Gartner. 2021. *Gartner Says Cloud Will Be the Centerpiece of New Digital Experiences*. <https://www.gartner.com/en/newsroom/press-releases/2021-11-10-gartner-says-cloud-will-be-the-centerpiece-of-new-digital-experiences>. Press Release, accessed 22 November 2025.
- Han, Haoyu, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A. Rossi, Subhabrata Mukherjee, dan Xianfeng Tang. 2024. "Retrieval-augmented generation with graphs (GraphRAG)". *arXiv preprint arXiv:2501.00309*.
- Hofer, Marvin, Daniel Obraczka, Alieh Saeedi, Hanna Köpcke, dan Erhard Rahm. 2024. "Construction of Knowledge Graphs: Current State and Challenges". *Information* 15 (8): 509.

- Ji, Ziwei, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, dan Pascale Fung. 2023. “Survey of Hallucination in Natural Language Generation”. *ACM Computing Surveys* 55 (12): 1–38.
- Knollmeyer, Simon, Oğuz Caymazer, dan Daniel Grossmann. 2025. “Document GraphRAG: Knowledge Graph Enhanced Retrieval Augmented Generation for Document Question Answering Within the Manufacturing Domain”. *Electronics* 14 (11): 2102. <https://doi.org/10.3390/electronics14112102>. <https://doi.org/10.3390/electronics14112102>.
- Kotstein, Sebastian, dan Christian Decker. 2024. “RESTBERTa: a Transformer-based question answering approach for semantic search in Web API documentation”. Published online 31 January 2024, *Cluster Computing*, <https://doi.org/10.1007/s10586-023-04237-x>.
- Liu, Zhijie, Anh Nguyen, Junjie Chen, Xin Zhang, Yanjie Li, dan Yingfei Xiong. 2024. “Deep Analysis of LLM-based Code Generation: Correctness, Complexity, and Security”. *IEEE Transactions on Software Engineering*, 1–20. <https://doi.org/10.1109/TSE.2024.3392499>.
- Ma, Chuangtao, Yongrui Chen, Tianxing Wu, Arijit Khan, dan Haofen Wang. 2025. “Large Language Models Meet Knowledge Graphs for Question Answering: Synthesis and Opportunities”. *arXiv preprint arXiv:2505.20099*.
- Niakšu, Olegas. 2015. “CRISP Data Mining Methodology Extension for Medical Domain”. *Baltic Journal of Modern Computing* 3 (2): 92–109. https://www.bjmc.lu.lv/fileadmin/user_upload/lu_portal/projekti/bjmc/Contents/3_2_2_Niaksu.pdf.
- Pan, Shirui, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, dan Xindong Wu. 2024. “Unifying Large Language Models and Knowledge Graphs: A Roadmap”. *IEEE Transactions on Knowledge and Data Engineering* 36 (7): 3580–3599. <https://doi.org/10.1109/TKDE.2024.3352100>.
- Rahman, A., dkk. 2023. “Empirical Analysis of Security Misconfigurations in Open-Source Kubernetes Manifests”. *Journal of Cloud Computing and Security* 12 (4): 112–128.

- Soldani, Jacopo, Damian Andrew Tamburri, dan Willem-Jan Van Den Heuvel. 2018. “The pains and gains of microservices: A Systematic grey literature review”. *Journal of Systems and Software* 146:215–232. <https://doi.org/10.1016/j.jss.2018.09.082>.
- The Kubernetes Authors. 2024a. *Kubernetes Concepts*. Diakses pada: 2024-05-15. <https://kubernetes.io/docs/concepts/>.
- . 2024b. “Kubernetes Documentation”. Diakses pada February 22, 2025. <https://kubernetes.io/docs/home/>.
- Wan, Yuwei, Zheyuan Chen, Ying Liu, Chong Chen, dan Michael Packianather. 2025. “Empowering LLMs by hybrid retrieval-augmented generation for domain-centric Q&A in smart manufacturing”. *Advanced Engineering Informatics* 65:103212. ISSN: 1474-0346. <https://doi.org/10.1016/j.aei.2025.103212>.
- Yu, Jun, Yintie Zhang, Yu Wu, dan Linhui Mao. 2021. “Research on the Practical Application of Visual Knowledge Graph in Technology Service Model and Intelligent Supervision”. *Journal of Physics: Conference Series* 1982:012040.
- Zhao, Penghao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, dan Bin Cui. 2024. “Retrieval-Augmented Generation for AI-Generated Content: A Survey”. *arXiv preprint arXiv:2402.19473*.

LAMPIRAN A

KODE PROGRAM

Pada umumnya, kode program tidak perlu disertakan di dalam laporan tugas akhir karena kode program biasanya sangat panjang dan sudah disimpan dalam bentuk berkas-berkas elektronik terpisah di repositori daring. Namun, jika ada bagian kode program singkat yang penting untuk ditulis dalam laporan tugas akhir, bagian tersebut dapat disertakan di dalam laporan.

A.1 Perangkat Lunak untuk Akuisisi Data dari Sensor Ultrasonik

```
1 % -- Example of pseudocode and source code listing --
2 % -- Gunakan minipage agar listing tidak terpotong ke halaman
   berikutnya --
3
4
5
6 \begin{minipage}{\textwidth}
7 \begin{lstlisting}[caption={Iterative binary search in Python},
   label={lst:binary_iter}, frame=single, captionpos=t, language=
   Python]
8 def binary_search(arr, target):
9     """
10     Performs binary search on a sorted array.
11
12     Args:
13         arr: A sorted list of elements
14         target: The element to search for
15
16     Returns:
17         Index of target if found, -1 otherwise
18     """
19     left = 0
20     right = len(arr) - 1
21
22     while left <= right:
23         mid = (left + right) // 2
24
```

```

25     if arr[mid] == target:
26         return mid
27     elif arr[mid] < target:
28         left = mid + 1
29     else:
30         right = mid - 1
31
32     return -1
33
34
35 \end{lstlisting}
36 \end{minipage}

```

Listing A.1 Binary search code

A.2 Perangkat Lunak untuk Pengolahan Data Akuisisi dan Visualisasi Hasil Pengukuran Jarak

asdjfkjashdkjfas

LAMPIRAN B

HASIL SURVEI LAPANGAN

B.1 Foto Survei Lokasi 1

Teks survei lokasi 1.

B.2 Foto Survei Lokasi 2

Teks survei lokasi 2.

B.3 Transkrip Wawancara dengan Narasumber

Teks transkrip wawancara.

